Fórum de Discussão

Projeto de Design Patterns — Paradigmas de Programação, turma CC5AN

Aplicação cliente/servidor: servidor **Flask** expondo a API e cliente gráfico em **Qt (PySide6)**, com nove padrões de projeto do catálogo GoF aplicados ao domínio de um fórum de discussão.

---

1. Visão geral

O sistema é um fórum de discussão dividido em dois módulos independentes:

- **servidor** — aplicação Flask que guarda o estado do fórum em memória e expõe um endpoint para cada recurso (usuários, tópicos, respostas, anexos, votos, inscrições, estado, formatação de conteúdo, busca e estatísticas);
- **cliente** — aplicação gráfica em PySide6 que consome esses endpoints. O cliente não contém regra de negócio: toda decisão acontece no servidor.

O domínio do fórum concentra nove padrões GoF, três de cada categoria. Para tornar o funcionamento deles visível durante a demonstração, **toda resposta do servidor traz um rastro** com as linhas dos padrões acionados naquela requisição, e o cliente exibe esse rastro em um painel lateral chamado *Padrões em ação*.

Nenhum banco de dados, API de terceiros, serviço de nuvem ou serviço de IA é utilizado. O estado vive na instância única de `Forum` (padrão Singleton) enquanto o servidor estiver em execução.

1.1 Padrões implementados

#	Padrão	Categoria	Arquivo	Papel no fórum
1	Singleton	Criacional	`forum/criacional/singleton.py`	Repositório único: usuários, tópicos e contador de identificadores
2	Factory Method	Criacional	`forum/criacional/factory_method.py`	Roteiro único de publicação que fabrica Tópico, Mensagem ou Anexo
3	Builder	Criacional	`forum/criacional/builder.py`	Montagem passo a passo de um tópico (título, autor, texto, tags, fixado)
4	Composite	Estrutural	`forum/estrutural/composite.py`	Árvore da discussão e sua serialização para JSON
5	Decorator	Estrutural	`forum/estrutural/decorator.py`	Filtros de conteúdo empilháveis: censura, destaque e assinatura
6	Facade	Estrutural	`forum/estrutural/facade.py`	Camada de serviço chamada pelas rotas Flask
7	Observer	Comportamental	`forum/comportamental/observer.py`	Notificação dos inscritos a cada nova resposta
8	Strategy	Comportamental	`forum/comportamental/strategy.py`	Ordenação da listagem de tópicos
9	State	Comportamental	`forum/comportamental/state.py`	Ciclo de vida do tópico: Aberto, Fechado e Arquivado

No lado do cliente, a classe `ApiForum` funciona como um **proxy remoto**: expõe os mesmos métodos da fachada do servidor, mas cada chamada é empacotada em uma requisição HTTP.

1.2 Estrutura de diretórios

```
forum-design-patterns/
├── requirements.txt              Flask, requests e PySide6
├── README.md
├── servidor/                     MÓDULO SERVIDOR
│   ├── app.py                    rotas Flask e envelope das respostas
│   └── __main__.py               python -m servidor
├── cliente/                      MÓDULO CLIENTE
│   ├── api.py                    ApiForum: proxy remoto dos endpoints
│   ├── dialogos.py               caixas de diálogo
│   ├── janela.py                 JanelaPrincipal (QMainWindow)
│   └── __main__.py               python -m cliente
├── forum/                        DOMÍNIO (compartilhado pelo servidor)
│   ├── modelos.py                Usuario
│   ├── rastro.py                 rastro dos padrões por requisição
│   ├── erros.py                  ErroDeForum
│   ├── criacional/               singleton.py, factory_method.py, builder.py
│   ├── estrutural/               composite.py, decorator.py, facade.py
│   └── comportamental/           observer.py, strategy.py, state.py
├── testes/
│   ├── test_padroes.py           33 testes do domínio
```

```
│   ├── test_servidor.py          35 testes dos endpoints
│   ├── test_cliente.py           12 testes do acesso do cliente
│   ├── test_cliente_gui.py       10 testes da janela gráfica
│   └── demonstracao.py           roteiro da aplicação em uso
└── docs/
    ├── DOCUMENTACAO.md           este documento
    ├── gerar_diagramas.py        regera os diagramas
    └── imagens/                  diagramas e capturas de tela
```

1.3 Como executar

```
pip install -r requirements.txt

python -m servidor        # terminal 1: servidor em http://127.0.0.1:5000
python -m cliente         # terminal 2: cliente gráfico
```

O cliente aceita outro endereço como argumento: `python -m cliente http://127.0.0.1:5050`.

Demais comandos:

```
python -m unittest discover -s testes -v     # 90 testes automatizados
python testes/demonstracao.py                # roteiro da aplicação em uso
python testes/demonstracao.py --sem-gui      # somente o roteiro HTTP
python docs/gerar_diagramas.py               # regera os diagramas de classes
```

Requisitos de ambiente: Python 3.9 ou superior e as três dependências de `requirements.txt`.

---

2. Requisitos

2.1 Requisitos funcionais

ID	Requisito	Endpoint	Onde aparece no cliente
RF01	Cadastrar participante com apelido, e-mail e papel (membro ou moderador)	`POST /api/usuarios`	Botão *Cadastrar usuário*
RF02	Listar os participantes cadastrados	`GET /api/usuarios`	Seletor *Usuário atual*
RF03	Abrir um tópico informando título, texto, tags e sinalização de fixado	`POST /api/topicos`	Botão *Abrir novo tópico*
RF04	Listar os tópicos com ordenação escolhida pelo usuário	`GET /api/topicos?ordenacao=`	Seletor *Ordenar por*
RF05	Exibir a árvore completa de uma discussão, em qualquer profundidade	`GET /api/topicos/<id>`	Painel central em árvore
RF06	Responder a um tópico ou a uma mensagem já existente	`POST /api/itens/<id>/respostas`	Botão *Responder*
RF07	Anexar arquivo a uma mensagem	`POST /api/mensagens/<id>/anexos`	Botão *Anexar arquivo*
RF08	Registrar voto em uma mensagem	`POST /api/mensagens/<id>/votos`	Botão *Votar*
RF09	Inscrever um participante em um tópico	`POST /api/topicos/<id>/inscricoes`	Botão *Inscrever usuário atual*
RF10	Notificar os inscritos a cada nova resposta	— (efeito de RF06)	*Fórum → Notificações*
RF11	Abrir, fechar ou arquivar um tópico	`PUT /api/topicos/<id>/estado`	Seletor de estado + *Aplicar*
RF12	Aplicar filtros combináveis a um texto antes de publicá-lo	`POST /api/conteudo/formatado`	*Conteúdo → Formatar texto*
RF13	Buscar por texto em todas as árvores de discussão	`GET /api/busca?termo=`	Campo de busca
RF14	Apresentar os números gerais do fórum	`GET /api/estatisticas`	*Fórum → Estatísticas*
RF15	Informar, a cada operação, quais padrões de projeto foram acionados	campo `rastro` de toda resposta	Painel *Padrões em ação*

2.2 Requisitos não-funcionais

ID	Requisito
RNF01	A aplicação roda sem banco de dados, API de terceiros, serviço de nuvem ou serviço de IA; o estado vive em memória no processo do servidor
RNF02	O repositório é executável após `pip install -r requirements.txt`, sem nenhuma outra configuração
RNF03	Servidor e cliente são módulos independentes e comunicam-se apenas por HTTP com corpo JSON
RNF04	O cliente não contém regra de negócio: toda validação e toda decisão ocorrem no servidor
RNF05	Toda resposta segue o mesmo envelope `{"dados": ..., "rastro": [...]}`, e todo erro devolve `{"erro": ..., "rastro": [...]}` com o status HTTP adequado
RNF06	O domínio não depende do Flask nem do Qt, e pode ser testado isoladamente
RNF07	Cada padrão de projeto fica em seu próprio módulo, com intenção, aplicação e participantes documentados no cabeçalho
RNF08	A suíte automatizada cobre domínio, endpoints, acesso do cliente e interface gráfica, e roda sem abrir janelas
RNF09	O cliente informa falha de comunicação com o servidor em linguagem compreensível, sem encerrar a aplicação
RNF10	Código, mensagens de interface e documentação em português

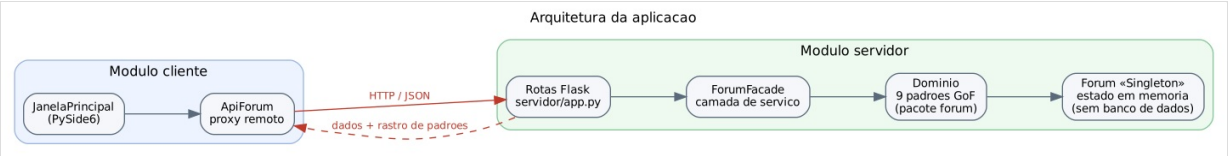
2.3 Condições de aceitação

ID	Condição	Verificado por
CA01	Subir o servidor e o cliente em duas linhas de comando, sem configuração adicional	Seção 1.3
CA02	Dois participantes com o mesmo apelido não são aceitos; a segunda tentativa devolve HTTP 409	`TesteUsuarios.test_apelido_duplicado`
CA03	Somente moderadores conseguem fixar um tópico; um membro recebe HTTP 403	`TesteTopicos.test_membro_nao_fixa` / `test_moderador_fixa`
CA04	Uma resposta a uma mensagem aparece aninhada sob ela na árvore, e não no nível do tópico	`TesteRespostasEAnexos.test_resposta_aninhada`
CA05	Um anexo não aceita filhos: a tentativa devolve HTTP 400	`TesteRespostasEAnexos.test_anexo_em_anexo_e_recusado`
CA06	Tópico fechado recusa novas respostas com HTTP 409	`TesteEstadoEObserver.test_fechar_bloqueia_resposta`
CA07	Tópico arquivado não pode ser reaberto, e permanece arquivado	`TesteEstadoEObserver.test_arquivado_nao_reabre`
CA08	Quem está inscrito em um tópico recebe aviso na caixa de entrada a cada nova resposta	`TesteEstadoEObserver.test_inscricao_e_notificacao`
CA09	Trocar a ordenação muda a ordem da listagem sem alterar os dados	`TesteTopicos.test_ordenacao_por_votos`
CA10	Os filtros de conteúdo são aplicados de dentro para fora e podem ser combinados	`TesteConteudoEConsultas.test_formatacao_com_pilha_de_filtros`
CA11	A busca encontra itens em qualquer nível da árvore e informa o caminho	`TesteConteudoEConsultas.test_busca_encontra_na_arvore`
CA12	Toda resposta traz o rastro dos padrões, e o rastro de uma requisição não vaza para a seguinte	`TesteEnvelope`
CA13	A árvore exibida pelo cliente reflete a estrutura recebida do servidor	`TesteJanelaPrincipal.test_arvore_reflete_a_estrutura_recebida`
CA14	Com o servidor fora do ar, o cliente avisa o usuário e continua aberto	`TesteJanelaPrincipal.test_falha_de_servidor_mostra_aviso`
CA15	A suíte automatizada completa passa sem falhas	Seção 8

---

3. Arquitetura

O cliente conversa apenas com `ApiForum`; `ApiForum` conversa apenas com os endpoints; as rotas conversam apenas com `ForumFacade`; e a fachada coordena os nove padrões. Cada camada conhece somente a seguinte.

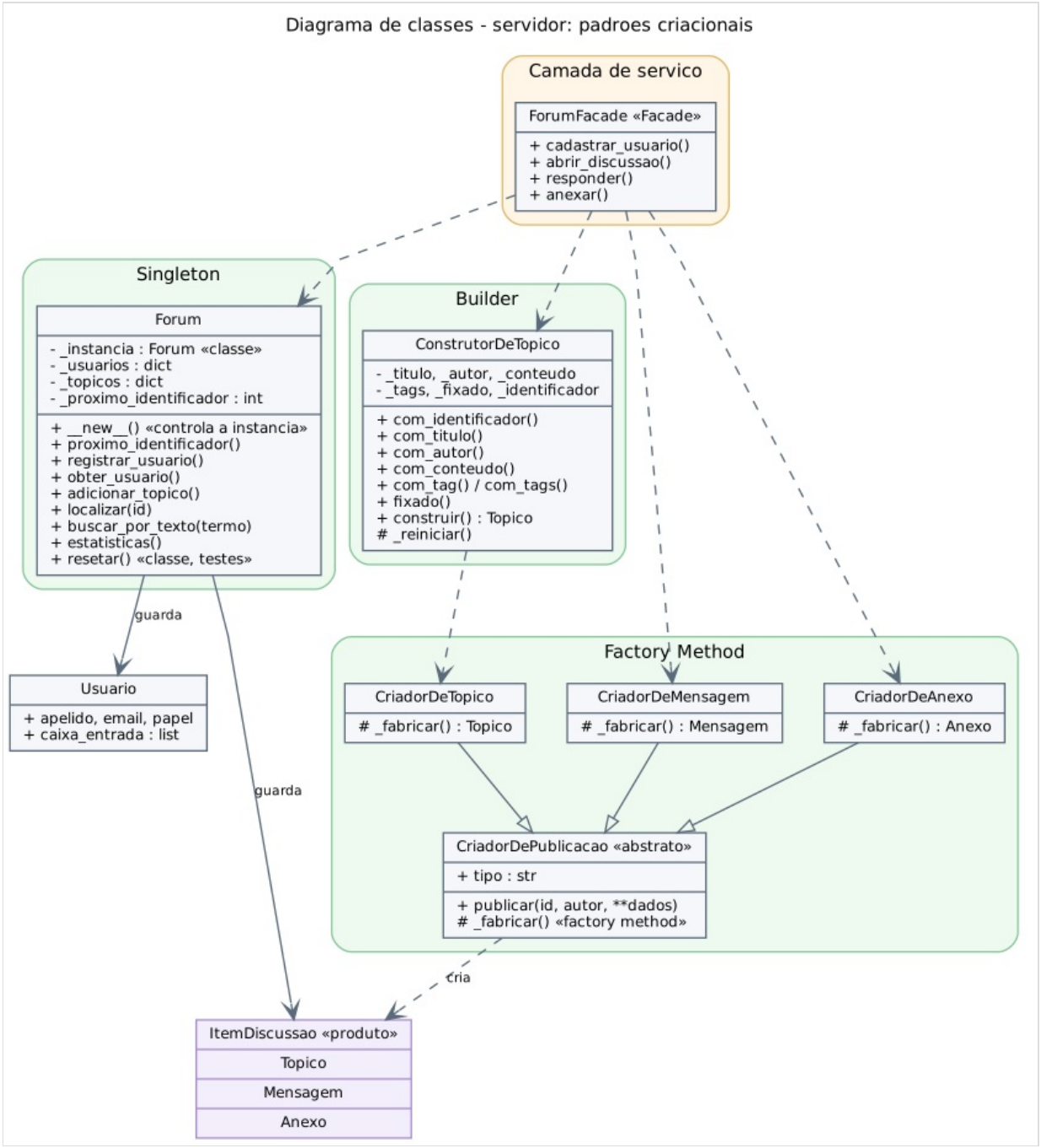


Arquitetura da aplicação

3.1 Diagramas de classes — servidor

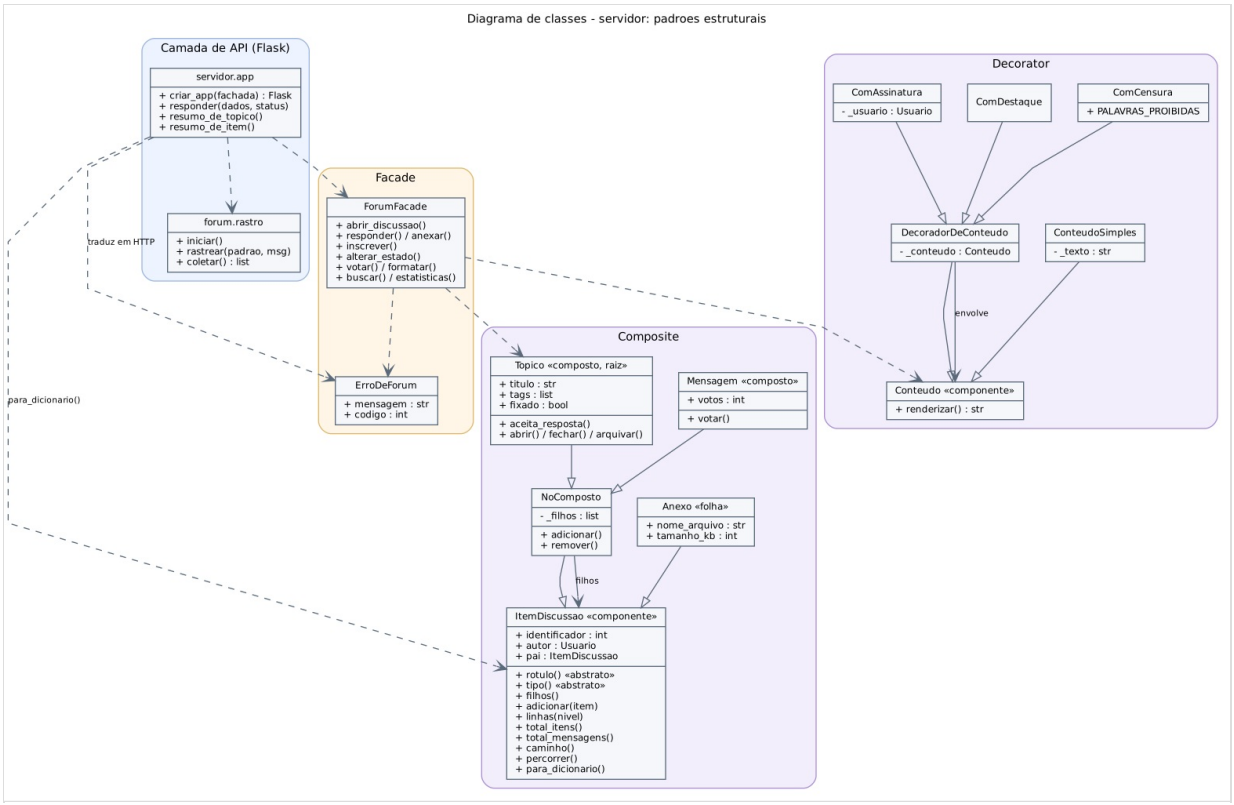
O servidor é apresentado em três diagramas, um por categoria de padrão, para que as assinaturas fiquem legíveis. A visão completa, com os nove padrões em um único diagrama, está em `docs/imagens/servidor-classes.png` e na versão vetorial `servidor-classes.svg`.

Padrões criacionais — o repositório único, a montagem do tópico e a fabricação dos produtos:



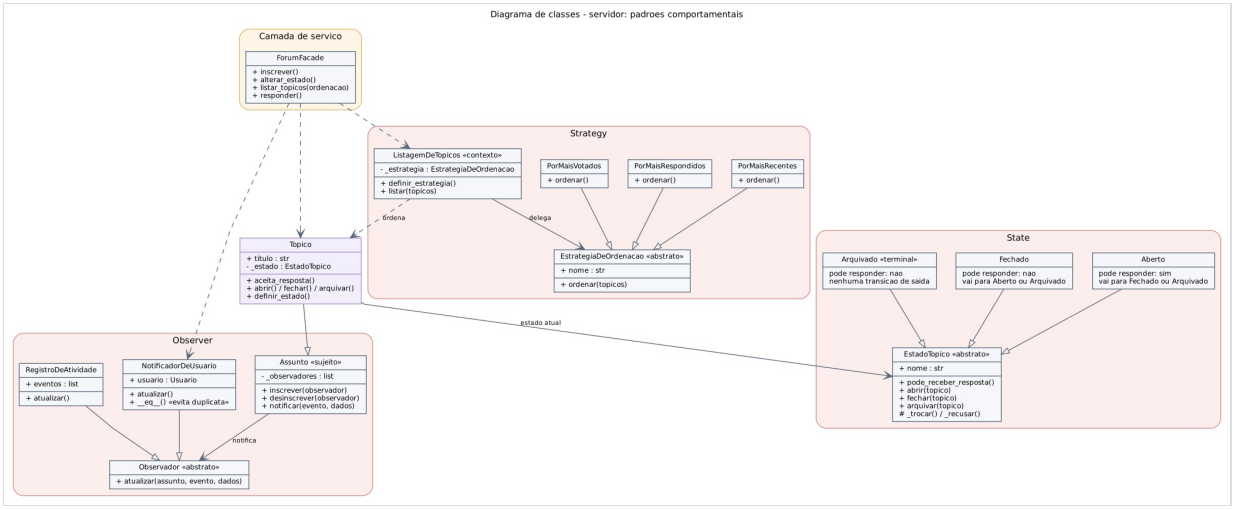
Padrões criacionais do servidor

Padrões estruturais — a camada de API, a fachada, a árvore da discussão e os filtros de conteúdo:



Padrões estruturais do servidor

Padrões comportamentais — notificações, ordenação e ciclo de vida do tópico:



Padrões comportamentais do servidor

3.2 Diagrama de classes — cliente

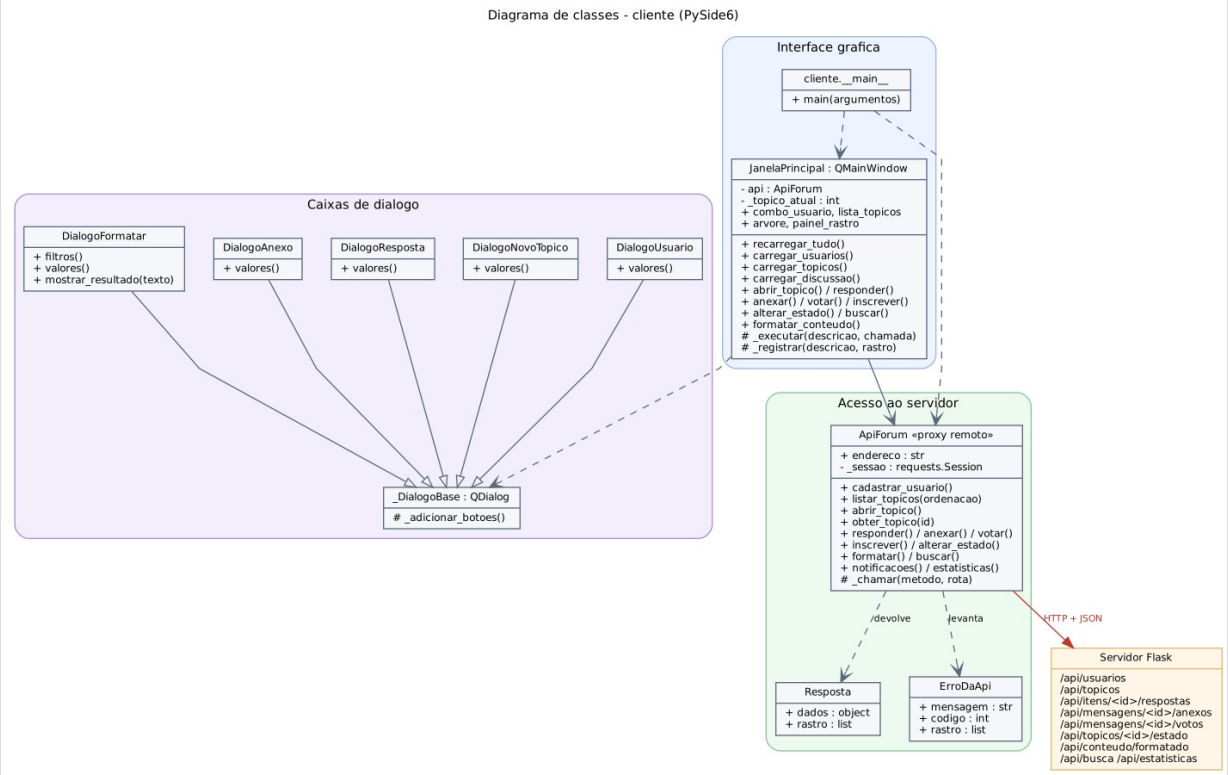


Diagrama de classes do cliente

---

4. Os nove padrões aplicados ao fórum

4.1 Singleton (criacional)

Intenção. Garantir que uma classe tenha uma única instância e fornecer um ponto global de acesso a ela.

Problema no fórum. O cadastro de usuários, a coleção de tópicos e o contador de identificadores precisam ser únicos no processo do servidor. Se cada requisição criasse o seu próprio objeto `Forum`, dois tópicos poderiam receber o identificador `#1` e um usuário cadastrado em uma chamada não existiria na seguinte.

Implementação. O controle fica em `__new__`: a primeira chamada cria o objeto e o guarda em `Forum._instancia`; as seguintes devolvem o mesmo objeto. O método de classe `resetar()` existe apenas para isolar os testes automatizados, que precisam de um fórum limpo a cada caso.

Como verificar. O endpoint `GET /api/estatisticas` devolve o campo `instancia_unica`, calculado comparando `Forum()` com a instância que a fachada já usa.

4.2 Factory Method (criacional)

Intenção. Definir uma interface para criar um objeto, deixando que as subclasses decidam qual classe instanciar.

Problema no fórum. Publicar um item segue sempre o mesmo roteiro — anunciar, fabricar, confirmar — mas o produto varia: `Topico`, `Mensagem` ou `Anexo`.

Implementação. `CriadorDePublicacao.publicar()` concentra o roteiro comum e chama `_fabricar()`, o Factory Method, sobrescrito por `CriadorDeTopico`, `CriadorDeMensagem` e `CriadorDeAnexo`. O dicionário `CRIADORES` permite obter o criador pelo nome do tipo, sem encadeamento de `if`.

4.3 Builder (criacional)

Intenção. Separar a construção de um objeto complexo da sua representação.

Problema no fórum. Abrir um tópico envolve cinco informações, sendo três opcionais (texto de abertura, tags e sinalização de fixado). Um construtor com cinco parâmetros posicionais seria ilegível e frágil, e o corpo JSON da requisição pode omitir qualquer uma delas.

Implementação. `ConstrutorDeTopico` expõe métodos encadeáveis (`com_titulo`, `com_autor`, `com_tags`, `fixado`), valida o que

é obrigatório em `construir()` e só então delega a fabricação ao Factory Method. Depois de entregar o produto, o construtor se reinicia, evitando que sobras de uma requisição vazem para a próxima.

4.4 Composite (estrutural)

Intenção. Compor objetos em estruturas de árvore e tratar objetos individuais e composições de maneira uniforme.

Problema no fórum. Uma discussão é uma árvore: o tópico contém mensagens, cada mensagem contém respostas e anexos, sem limite de profundidade. Exibir, contar, percorrer e **serializar para JSON** precisam funcionar igual em qualquer nível.

Implementação. `ItemDiscussao` é o componente e define `linhas()`, `total_itens()`, `caminho()`, `percorrer()` e `para_dicionario()` de forma recursiva. `Anexo` é a folha e recusa filhos com `TypeError`; `Mensagem` e `Topico` são compostos. O método `caminho()` reconstrói o endereço do nó até a raiz (por exemplo `/T1/M3/M4/A5`) usando a referência ao pai.

No cliente. `JanelaPrincipal._montar_no()` percorre o mesmo JSON de forma recursiva para montar o `QTreeWidget`: a uniformidade do Composite atravessa a fronteira HTTP.

4.5 Decorator (estrutural)

Intenção. Anexar responsabilidades a um objeto dinamicamente, como alternativa flexível à herança.

Problema no fórum. O texto publicado pode passar por censura de palavras proibidas, destaque de moderação e assinatura do autor, em qualquer combinação e ordem. Criar uma subclasse para cada combinação seria inviável.

Implementação. `Conteudo` é o componente, `ConteudoSimples` guarda o texto cru e `DecoradorDeConteudo` envolve outro `Conteudo`. Os decoradores concretos são empilhados em tempo de execução, conforme a lista de filtros que chega no corpo da requisição. A ordem da pilha altera o resultado final, como demonstram a seção 9 e o teste `test_ordem_importa`.

4.6 Facade (estrutural)

Intenção. Fornecer uma interface unificada para um conjunto de interfaces de um subsistema.

Problema no fórum. Abrir uma discussão exige coordenar seis participantes: repositório único, construtor, criador, árvore, observadores e estado inicial. Uma rota HTTP não deve conhecer nada disso.

Implementação. `ForumFacade` oferece operações de alto nível (`abrir_discussao`, `responder`, `buscar`, `alterar_estado`) e esconde a coordenação. É a camada de serviço do servidor: cada rota chama um método da fachada e traduz o resultado em JSON. Quando uma regra é violada, a fachada levanta `ErroDeForum` com o status HTTP adequado — o domínio não imprime mensagens nem importa nada do Flask.

4.7 Observer (comportamental)

Intenção. Definir uma dependência um-para-muitos, de modo que a mudança de estado de um objeto notifique automaticamente seus dependentes.

Problema no fórum. Quem acompanha um tópico precisa ser avisado a cada nova resposta, e o tópico não pode depender da forma de entrega do aviso.

Implementação. `Topico` herda de `Assunto` e mantém a lista de observadores. `NotificadorDeUsuario` entrega o aviso na caixa de entrada de um usuário; `RegistroDeAtividade` grava o evento para auditoria. Os dois implementam a mesma interface `Observador`, e o tópico não sabe a diferença entre eles. A comparação por `__eq__` impede inscrições duplicadas do mesmo usuário.

4.8 Strategy (comportamental)

Intenção. Definir uma família de algoritmos, encapsulá-los e torná-los intercambiáveis.

Problema no fórum. A listagem de tópicos pode ser ordenada por mais recentes, mais respondidos ou mais votados, e a escolha chega na própria requisição, em `?ordenacao=`.

Implementação. Cada algoritmo é uma classe que implementa `ordenar()`. O contexto `ListagemDeTopicos` apenas delega à estratégia configurada e pode trocá-la a qualquer momento com `definir_estrategia()`, sem nenhum `if` sobre o tipo de ordenação.

4.9 State (comportamental)

Intenção. Permitir que um objeto altere seu comportamento quando seu estado interno muda.

Problema no fórum. Um tópico aberto aceita respostas; fechado, não; arquivado, nunca mais volta atrás. Codificar isso com condicionais espalharia a mesma regra por várias rotas.

Implementação. `EstadoTopico` declara `pode_receber_resposta()` e as transições `abrir`, `fechar` e `arquivar`. Cada estado concreto autoriza apenas as transições válidas e recusa as demais; `Arquivado` é terminal. O tópico delega todas as decisões ao objeto de estado atual, e a fachada transforma uma transição recusada em HTTP 409.

Transições válidas.

```
Aberto    --fechar-->    Fechado
Aberto    --arquivar-->  Arquivado
Fechado   --abrir-->     Aberto
Fechado   --arquivar-->  Arquivado
Arquivado --(nenhuma)->  estado terminal
```

---

5. API do servidor

5.1 Envelope das respostas

Toda resposta bem-sucedida tem o mesmo formato:

```
{
  "dados": { "identificador": 1, "titulo": "Como funciona a recursao em Python?" },
  "rastro": [
    "[Facade] Coordenando Builder + Factory Method + Singleton + Observer",
    "[Builder] Montando o topico passo a passo:",
    "[Factory Method] CriadorDeTopico vai fabricar um(a) Topico"
  ]
}
```

E todo erro:

```
{
  "erro": "O topico esta Fechado e nao aceita novas respostas.",
  "rastro": ["[Facade] Consultando o estado do topico #1"]
}
```

O campo `rastro` é limpo no início de cada requisição (`@app.before_request`) e coletado na resposta, de modo que o rastro de uma chamada nunca vaza para a seguinte. As mesmas linhas são gravadas no log do servidor.

5.2 Endpoints

Método	Rota	Corpo / parâmetros	Resposta	Padrões acionados
GET	`/`	—	Lista dos endpoints	—
GET	`/api/saude`	—	`{"status": "ok"}`	—
GET	`/api/usuarios`	—	Lista de usuários	—
POST	`/api/usuarios`	`apelido`, `email`, `papel`	201 com o usuário	Facade, Singleton
GET	`/api/usuarios/<apelido>/notificacoes`	—	Avisos da caixa de entrada	Observer
GET	`/api/topicos`	`?ordenacao=recentes|respondidos|votados`	Lista resumida	Strategy
POST	`/api/topicos`	`autor`, `titulo`, `conteudo`, `tags`, `fixado`	201 com a árvore	Builder, Factory Method, Observer
GET	`/api/topicos/<id>`	—	Árvore completa	Composite
POST	`/api/topicos/<id>/inscricoes`	`apelido`	201 com o tópico	Observer
PUT	`/api/topicos/<id>/estado`	`acao`	Tópico com o novo estado	State
POST	`/api/itens/<id>/respostas`	`autor`, `conteudo`	201 com a mensagem	State, Factory Method, Composite, Observer
POST	`/api/mensagens/<id>/anexos`	`autor`, `nome_arquivo`, `tamanho_kb`	201 com o	Factory Method,

			anexo	Composite
POST	`/api/mensagens/<id>/votos`	—	Mensagem com o total	Composite
POST	`/api/conteudo/formatado`	`texto`, `filtros`, `autor`	Texto renderizado	Decorator
GET	`/api/busca`	`?termo=`	Itens encontrados	Facade, Composite
GET	`/api/estatisticas`	—	Números do fórum	Singleton

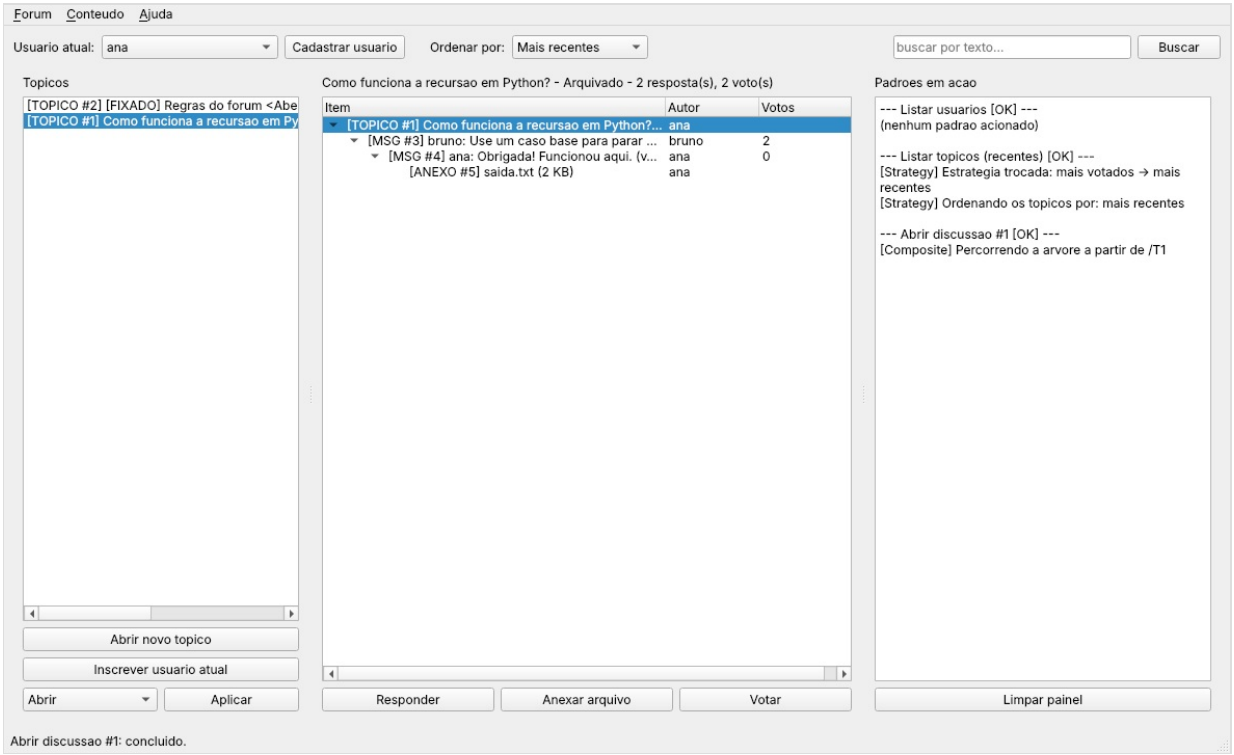
5.3 Códigos de status

Código	Quando ocorre	Exemplo
200	Consulta ou alteração bem-sucedida	`GET /api/topicos`
201	Recurso criado	`POST /api/topicos`
400	Entrada inválida	título vazio, filtro desconhecido, voto em tópico
403	Operação não permitida ao papel do usuário	membro tentando fixar um tópico
404	Recurso inexistente	usuário não cadastrado, tópico inexistente
405	Método não permitido para a rota	`DELETE /api/usuarios`
409	Regra de negócio violada	apelido repetido, resposta em tópico fechado

---

6. Cliente gráfico

A janela principal tem três colunas: a lista de tópicos à esquerda, a árvore da discussão selecionada ao centro e o painel *Padrões em ação* à direita, que acumula o rastro devolvido pelo servidor a cada requisição. A barra superior concentra o usuário atual, a ordenação da listagem (Strategy) e a busca; a barra inferior de cada coluna traz as ações.



Janela principal do cliente

A caixa de diálogo de formatação monta a pilha de decoradores: cada filtro marcado é mais um decorador envolvendo o texto, e o resultado vem renderizado pelo servidor.



Caixa de diálogo de formatação

Pontos de projeto do cliente:

- **Sem regra de negócio.** A janela nunca decide se uma operação é válida. Ela envia a requisição e trata a resposta: em caso de erro, mostra a mensagem que veio do servidor.
- **Um único ponto de entrada para erros.** O método `_executar()` envolve toda chamada à API, registra o rastro no painel, exibe o aviso e escreve na barra de status. Nenhuma ação da janela repete esse tratamento.
- **Falha de comunicação é tratada.** Com o servidor fora do ar, `ApiForum` levanta `ErroDaApi` com uma mensagem em português, e o cliente continua aberto.

---

7. Código-fonte

Esta seção traz o código da aplicação: domínio, servidor e cliente. Os arquivos de teste não são reproduzidos aqui — estão no repositório, em `testes/`, e são descritos na seção 8.

7.1 Domínio

Arquivo: `forum/modelos.py`

```
        """Modelos de dominio que nao pertencem a nenhum padrao especifico."""


        class Usuario:
            """Participante do forum.

            A caixa de entrada e preenchida pelos observadores (padrao Observer)
            sempre que um topico assinado pelo usuario recebe uma nova resposta.
            """

            PAPEIS = ("membro", "moderador")

            def __init__(self, apelido, email, papel="membro"):
                if papel not in self.PAPEIS:
                    raise ValueError(f'Papel invalido: "{papel}". Use um de {self.PAPEIS}.')
                self.apelido = apelido
                self.email = email
                self.papel = papel
                self.caixa_entrada = []

            @property
            def e_moderador(self):
                return self.papel == "moderador"

            def __str__(self):
```

```
            return f"{self.apelido} ({self.papel})"

        def __repr__(self):
            return f"Usuario({self.apelido!r}, {self.email!r}, {self.papel!r})"
```

Arquivo: `forum/rastro.py`

```
    """Rastro de padroes acionados durante uma operacao.

    O dominio nao imprime nada na tela: cada padrao registra aqui a linha que
    descreve o que fez. O servidor limpa o rastro no inicio de cada requisicao
    e devolve as linhas coletadas junto com a resposta, de modo que o cliente
    grafico consegue mostrar, a cada acao, quais padroes entraram em cena.

    Como o servidor Flask atende requisicoes em threads diferentes, o rastro e
    guardado em armazenamento local de thread.
    """

    import threading

    _local = threading.local()


    def iniciar():
        """Zera o rastro da thread atual. Chamado no inicio de cada requisicao."""
        _local.linhas = []


    def rastrear(padrao, mensagem):
        """Registra uma linha de rastro e a devolve.

        `padrao` e o nome do padrao de projeto responsavel pela acao (ou None
        para linhas de apoio, como os passos internos do Builder).
        """
        linha = f"[{padrao}] {mensagem}" if padrao else mensagem
        linhas = getattr(_local, "linhas", None)
        if linhas is None:
            linhas = []
            _local.linhas = linhas
        linhas.append(linha)
        return linha


    def linhas():
        """Devolve as linhas acumuladas sem apagar o rastro."""
        return list(getattr(_local, "linhas", []))


    def coletar():
        """Devolve as linhas acumuladas e zera o rastro."""
        acumuladas = linhas()
        _local.linhas = []
        return acumuladas
```

Arquivo: `forum/erros.py`

```
    """Excecao de dominio compartilhada pelo servidor e pelos testes."""


    class ErroDeForum(Exception):
        """Regra de negocio violada.

        O atributo `codigo` carrega o status HTTP que o servidor deve devolver,
        mas o dominio nao depende do Flask: quem traduz a excecao em resposta e
        a camada de API.
        """

        def __init__(self, mensagem, codigo=400):
            super().__init__(mensagem)
            self.mensagem = mensagem
            self.codigo = codigo
```

Arquivo: `forum/criacional/singleton.py`

```
    """Padrao SINGLETON (criacional).

    Intencao
        Garantir que uma classe tenha uma unica instancia e fornecer um ponto
        global de acesso a ela.

    Aplicacao no forum
        Existe um unico forum em execucao. Ele concentra o cadastro de
        usuarios, a colecao de topicos e o contador de identificadores. Se
        cada parte do sistema criasse o seu proprio "Forum", os contadores e
        as colecoes ficariam dessincronizados.

    Implementacao
        O controle e feito em __new__: a primeira chamada cria e guarda a
        instancia em Forum._instancia; as seguintes devolvem o mesmo objeto.
    """


    from ..rastro import rastrear
```

```
class Forum:
    """Repositorio central do forum. Instancia unica."""

    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            rastrear("Singleton", "Primeira chamada: criando a unica instancia de Forum")
            cls._instancia = super().__new__(cls)
            cls._instancia._inicializar()
        return cls._instancia

    def _inicializar(self):
        self.nome = "Forum de Discussao"
        self._usuarios = {}
        self._topicos = {}
        self._proximo_identificador = 1

    # --- contador de identificadores ------------------------------------
    def proximo_identificador(self):
        atual = self._proximo_identificador
        self._proximo_identificador += 1
        return atual

    # --- usuarios -------------------------------------------------------
    def registrar_usuario(self, usuario):
        if usuario.apelido in self._usuarios:
            return False
        self._usuarios[usuario.apelido] = usuario
        return True

    def obter_usuario(self, apelido):
        return self._usuarios.get(apelido)

    @property
    def usuarios(self):
        return tuple(self._usuarios.values())

    # --- topicos --------------------------------------------------------
    def adicionar_topico(self, topico):
        self._topicos[topico.identificador] = topico
        return topico

    def obter_topico(self, identificador):
        return self._topicos.get(identificador)

    @property
    def topicos(self):
        return tuple(self._topicos.values())

    # --- consultas sobre a arvore (Composite) ---------------------------
    def localizar(self, identificador):
        """Procura um item (topico, mensagem ou anexo) em todas as arvores."""
        for topico in self._topicos.values():
            for item in topico.percorrer():
                if item.identificador == identificador:
                    return item
        return None

    def buscar_por_texto(self, termo):
        """Devolve todos os itens cujo conteudo ou titulo contem o termo."""
        termo = termo.lower()
        encontrados = []
        for topico in self._topicos.values():
            for item in topico.percorrer():
                alvo = f"{getattr(item, 'titulo', '')} {item.conteudo}".lower()
                if termo in alvo:
                    encontrados.append(item)
        return encontrados

    # --- estatisticas ---------------------------------------------------
    def estatisticas(self):
        topicos = self.topicos
        return {
            "usuarios": len(self._usuarios),
            "topicos": len(topicos),
            "mensagens": sum(t.total_mensagens() for t in topicos),
            "itens_na_arvore": sum(t.total_itens() for t in topicos),
            "votos": sum(t.total_votos() for t in topicos),
            "proximo_identificador": self._proximo_identificador,
        }

    @classmethod
    def resetar(cls):
        """Descarta a instancia unica. Usado apenas pelos testes."""
        cls._instancia = None

    def __repr__(self):
        return f"<Forum id=0x{id(self):x} topicos={len(self._topicos)}>"
```

Arquivo: `forum/criacional/factory_method.py`

```
"""Padrao FACTORY METHOD (criacional).

Intencao
    Definir uma interface para criar um objeto, mas deixar que as
    subclasses decidam qual classe instanciar. O Factory Method permite
```

```
        que uma classe delegue a instanciacao as subclasses.

    Aplicacao no forum
        Publicar um item sempre segue o mesmo roteiro (anunciar, fabricar,
        confirmar). O que muda e o produto: um Topico, uma Mensagem ou um
        Anexo. O metodo publicar() e o codigo comum; _fabricar() e o Factory
        Method sobrescrito por cada criador concreto.

    Participantes
        Criador (Creator)           -> CriadorDePublicacao
        Criadores concretos         -> CriadorDeTopico, CriadorDeMensagem, CriadorDeAnexo
        Produto (Product)           -> ItemDiscussao
        Produtos concretos          -> Topico, Mensagem, Anexo
    """

    from abc import ABC, abstractmethod

    from ..estrutural.composite import Anexo, Mensagem, Topico
    from ..rastro import rastrear


    class CriadorDePublicacao(ABC):
        """Criador: implementa o roteiro de publicacao e declara o Factory Method."""

        tipo = "publicacao"

        @abstractmethod
        def _fabricar(self, identificador, autor, **dados):
            """Factory Method: cada subclasse devolve o seu produto concreto."""

        def publicar(self, identificador, autor, **dados):
            rastrear("Factory Method", f"{type(self).__name__} vai fabricar um(a) {self.tipo}")
            produto = self._fabricar(identificador, autor, **dados)
            rastrear("Factory Method", f"Produto criado: {produto.rotulo()}")
            return produto


    class CriadorDeTopico(CriadorDePublicacao):
        tipo = "Topico"

        def _fabricar(self, identificador, autor, **dados):
            return Topico(
                identificador,
                autor,
                dados["titulo"],
                dados.get("conteudo", ""),
                dados.get("tags"),
            )


    class CriadorDeMensagem(CriadorDePublicacao):
        tipo = "Mensagem"

        def _fabricar(self, identificador, autor, **dados):
            return Mensagem(identificador, autor, dados["conteudo"])


    class CriadorDeAnexo(CriadorDePublicacao):
        tipo = "Anexo"

        def _fabricar(self, identificador, autor, **dados):
            return Anexo(
                identificador,
                autor,
                dados["nome_arquivo"],
                dados.get("tamanho_kb", 0),
            )


    CRIADORES = {
        "topico": CriadorDeTopico,
        "mensagem": CriadorDeMensagem,
        "anexo": CriadorDeAnexo,
    }


    def obter_criador(tipo):
        """Devolve uma instancia do criador registrado para o tipo informado."""
        if tipo not in CRIADORES:
            raise ValueError(f'Tipo de publicacao desconhecido: "{tipo}".')
        return CRIADORES[tipo]()
```

Arquivo: `forum/criacional/builder.py`

```
    """Padrao BUILDER (criacional).

    Intencao
        Separar a construcao de um objeto complexo da sua representacao, de
        modo que o mesmo processo de construcao possa criar representacoes
        diferentes.

    Aplicacao no forum
        Abrir um topico envolve muitas partes opcionais: titulo, autor, texto
        de abertura, lista de tags e sinalizacao de fixado. Um construtor com
        cinco parametros opcionais seria ilegivel; o Builder monta o topico
        passo a passo, valida o que e obrigatorio e so entao chama o Factory
        Method para fabricar o produto final.
```

```
        Participantes
            Builder / Builder concreto -> ConstrutorDeTopico
            Produto (Product)          -> Topico
            Diretor (Director)         -> ForumFacade (forum.estrutural.facade)
        """

        from ..rastro import rastrear
        from .factory_method import CriadorDeTopico


        class ConstrutorDeTopico:
            """Monta um Topico passo a passo. Todos os metodos sao encadeaveis."""

            def __init__(self):
                self._reiniciar()

            def _reiniciar(self):
                self._identificador = None
                self._titulo = None
                self._autor = None
                self._conteudo = ""
                self._tags = []
                self._fixado = False

            # --- passos de construcao ------------------------------------------------
            def com_identificador(self, identificador):
                self._identificador = identificador
                return self

            def com_titulo(self, titulo):
                titulo = (titulo or "").strip()
                if not titulo:
                    raise ValueError("O titulo do topico nao pode ser vazio.")
                self._titulo = titulo
                return self

            def com_autor(self, autor):
                self._autor = autor
                return self

            def com_conteudo(self, conteudo):
                self._conteudo = conteudo or ""
                return self

            def com_tag(self, tag):
                tag = (tag or "").strip()
                if tag and tag not in self._tags:
                    self._tags.append(tag)
                return self

            def com_tags(self, tags):
                for tag in tags or []:
                    self.com_tag(tag)
                return self

            def fixado(self, valor=True):
                self._fixado = valor
                return self

            # --- entrega do produto --------------------------------------------------
            def construir(self):
                if self._identificador is None:
                    raise ValueError("Builder incompleto: identificador nao informado.")
                if self._autor is None:
                    raise ValueError("Builder incompleto: autor nao informado.")
                if self._titulo is None:
                    raise ValueError("Builder incompleto: titulo nao informado.")

                rastrear("Builder", "Montando o topico passo a passo:")
                rastrear(None, f"  - titulo ...: {self._titulo}")
                rastrear(None, f"  - autor ....: {self._autor.apelido}")
                rastrear(None, f"  - conteudo .: {self._conteudo or '(vazio)'}")
                rastrear(None, f"  - tags .....: {', '.join(self._tags) if self._tags else '(nenhuma)'}")
                rastrear(None, f"  - fixado ...: {'sim' if self._fixado else 'nao'}")

                topico = CriadorDeTopico().publicar(
                    self._identificador,
                    self._autor,
                    titulo=self._titulo,
                    conteudo=self._conteudo,
                    tags=self._tags,
                )
                topico.fixado = self._fixado
                self._reiniciar()
                return topico
```

Arquivo: `forum/estrutural/composite.py`

```
        """Padrao COMPOSITE (estrutural).

        Intencao
            Compor objetos em estruturas de arvore para representar hierarquias
            todo-parte, permitindo que o cliente trate objetos individuais e
            composicoes de objetos de maneira uniforme.

        Aplicacao no forum
            Uma discussao e uma arvore: o Topico contem Mensagens, cada Mensagem
            pode conter respostas (outras Mensagens) e Anexos. Exibir, contar e
```

```
    percorrer funcionam da mesma forma em qualquer nivel, porque todos os
    nos implementam a interface ItemDiscussao.

Participantes
    Componente (Component) -> ItemDiscussao
    Folha (Leaf)           -> Anexo
    Composto (Composite)   -> Mensagem, Topico
"""

from abc import ABC, abstractmethod

from ..comportamental.observer import Assunto
from ..comportamental.state import Aberto


class ItemDiscussao(ABC):
    """Componente: interface comum as folhas e aos compostos."""

    def __init__(self, identificador, autor, conteudo):
        self.identificador = identificador
        self.autor = autor
        self.conteudo = conteudo
        self.pai = None

    # --- interface uniforme ------------------------------------------------
    @property
    @abstractmethod
    def tag(self):
        """Prefixo curto usado na composicao do caminho (ex.: T1, M2, A3)."""

    @abstractmethod
    def rotulo(self):
        """Linha de texto que representa o item na arvore."""

    def filhos(self):
        """Folhas devolvem uma colecao vazia; compostos sobrescrevem."""
        return ()

    def adicionar(self, item):
        raise TypeError(f"{type(self).__name__} e uma folha e nao aceita filhos.")

    def remover(self, item):
        raise TypeError(f"{type(self).__name__} e uma folha e nao possui filhos.")

    # --- operacoes recursivas ----------------------------------------------
    def linhas(self, nivel=0):
        """Arvore achatada em linhas de texto indentadas, em pre-ordem."""
        resultado = ["    " * nivel + self.rotulo()]
        for filho in self.filhos():
            resultado.extend(filho.linhas(nivel + 1))
        return resultado

    def texto(self):
        return "\n".join(self.linhas())

    @property
    @abstractmethod
    def tipo(self):
        """Nome do tipo de no, usado na serializacao para o cliente."""

    def para_dicionario(self):
        """Serializa o no e todos os descendentes para JSON.

        A uniformidade do Composite aparece aqui: a mesma chamada funciona
        para o topico inteiro, para uma unica resposta ou para um anexo.
        """
        dados = {
            "identificador": self.identificador,
            "tipo": self.tipo,
            "tag": self.tag,
            "caminho": self.caminho(),
            "autor": self.autor.apelido,
            "conteudo": self.conteudo,
            "rotulo": self.rotulo(),
            "total_itens": self.total_itens(),
            "filhos": [filho.para_dicionario() for filho in self.filhos()],
        }
        dados.update(self._extras())
        return dados

    def _extras(self):
        """Campos adicionais de cada tipo de no."""
        return {}

    def total_itens(self):
        return 1 + sum(filho.total_itens() for filho in self.filhos())

    def total_mensagens(self):
        proprio = 1 if isinstance(self, Mensagem) else 0
        return proprio + sum(filho.total_mensagens() for filho in self.filhos())

    def total_votos(self):
        proprio = getattr(self, "votos", 0)
        return proprio + sum(filho.total_votos() for filho in self.filhos())

    def caminho(self):
        """Caminho do no a partir da raiz da discussao (ex.: /T1/M2/A3)."""
        if self.pai is None:
            return f"/{self.tag}"
        return f"{self.pai.caminho()}/{self.tag}"
```

```
    def percorrer(self):
        """Percurso em pre-ordem: o proprio no e depois os descendentes."""
        yield self
        for filho in self.filhos():
            yield from filho.percorrer()

    def raiz(self):
        return self if self.pai is None else self.pai.raiz()

    def __str__(self):
        return self.rotulo()


class Anexo(ItemDiscussao):
    """Folha: arquivo anexado a uma mensagem. Nao possui filhos."""

    def __init__(self, identificador, autor, nome_arquivo, tamanho_kb=0):
        super().__init__(identificador, autor, nome_arquivo)
        self.nome_arquivo = nome_arquivo
        self.tamanho_kb = tamanho_kb

    @property
    def tag(self):
        return f"A{self.identificador}"

    @property
    def tipo(self):
        return "anexo"

    def _extras(self):
        return {"nome_arquivo": self.nome_arquivo, "tamanho_kb": self.tamanho_kb}

    def rotulo(self):
        return f"[ANEXO #{self.identificador}] {self.nome_arquivo} ({self.tamanho_kb} KB)"


class NoComposto(ItemDiscussao):
    """Implementacao comum aos nos que armazenam filhos."""

    def __init__(self, identificador, autor, conteudo):
        super().__init__(identificador, autor, conteudo)
        self._filhos = []

    def filhos(self):
        return tuple(self._filhos)

    def adicionar(self, item):
        if not isinstance(item, ItemDiscussao):
            raise TypeError("Somente objetos ItemDiscussao podem ser adicionados.")
        item.pai = self
        self._filhos.append(item)
        return item

    def remover(self, item):
        if item not in self._filhos:
            return False
        self._filhos.remove(item)
        item.pai = None
        return True


class Mensagem(NoComposto):
    """Composto: resposta que pode conter outras respostas e anexos."""

    def __init__(self, identificador, autor, conteudo):
        super().__init__(identificador, autor, conteudo)
        self.votos = 0

    @property
    def tag(self):
        return f"M{self.identificador}"

    @property
    def tipo(self):
        return "mensagem"

    def _extras(self):
        return {"votos": self.votos}

    def votar(self):
        self.votos += 1
        return self.votos

    def rotulo(self):
        return (
            f"[MSG #{self.identificador}] {self.autor.apelido}: "
            f"{self.conteudo} (votos: {self.votos})"
        )


class Topico(NoComposto, Assunto):
    """Composto raiz da discussao.

    Acumula tres papeis de padroes diferentes:
      - Composite : e a raiz da arvore de mensagens;
      - Observer  : e o Assunto observado pelos usuarios inscritos;
      - State     : delega a um objeto EstadoTopico as regras de resposta.
    """
```

```
    def __init__(self, identificador, autor, titulo, conteudo, tags=None):
        NoComposto.__init__(self, identificador, autor, conteudo)
        Assunto.__init__(self)
        self.titulo = titulo
        self.tags = list(tags or [])
        self.fixado = False
        self._estado = Aberto()

    @property
    def tag(self):
        return f"T{self.identificador}"

    @property
    def tipo(self):
        return "topico"

    def _extras(self):
        return {
            "titulo": self.titulo,
            "tags": list(self.tags),
            "fixado": self.fixado,
            "estado": self._estado.nome,
            "aceita_resposta": self.aceita_resposta(),
            "total_respostas": self.total_respostas(),
            "total_votos": self.total_votos(),
            "inscritos": [
                observador.usuario.apelido
                for observador in self.observadores
                if hasattr(observador, "usuario")
            ],
        }

    # --- State ------------------------------------------------------------
    @property
    def estado(self):
        return self._estado

    def definir_estado(self, estado):
        self._estado = estado

    def aceita_resposta(self):
        return self._estado.pode_receber_resposta()

    def abrir(self):
        return self._estado.abrir(self)

    def fechar(self):
        return self._estado.fechar(self)

    def arquivar(self):
        return self._estado.arquivar(self)

    # --- Composite --------------------------------------------------------
    def total_respostas(self):
        return self.total_mensagens()

    def rotulo(self):
        marcador = "[FIXADO] " if self.fixado else ""
        etiquetas = f" (tags: {', '.join(self.tags)})" if self.tags else ""
        return (
            f"[TOPICO #{self.identificador}] {marcador}{self.titulo} "
            f"<{self._estado.nome}> - por {self.autor.apelido}{etiquetas}"
        )
```

Arquivo: `forum/estrutural/decorator.py`

```
"""Padrao DECORATOR (estrutural).

Intencao
    Anexar responsabilidades adicionais a um objeto dinamicamente. Os
    decoradores sao uma alternativa flexivel ao uso de heranca para
    estender funcionalidades.

Aplicacao no forum
    O texto publicado passa por filtros que podem ser combinados em
    qualquer ordem e quantidade: censura de palavras proibidas, destaque
    de moderador e assinatura do autor. Criar uma subclasse para cada
    combinacao possivel seria inviavel; os decoradores sao empilhados em
    tempo de execucao.

Participantes
    Componente (Component)   -> Conteudo
    Componente concreto      -> ConteudoSimples
    Decorador (Decorator)    -> DecoradorDeConteudo
    Decoradores concretos    -> ComCensura, ComDestaque, ComAssinatura
"""

import re
from abc import ABC, abstractmethod


class Conteudo(ABC):
    """Componente: tudo que sabe se renderizar como texto."""

    @abstractmethod
    def renderizar(self):
        """Devolve o texto final que sera exibido no forum."""
```

```
        def __str__(self):
            return self.renderizar()


    class ConteudoSimples(Conteudo):
        """Componente concreto: o texto cru digitado pelo usuario."""

        def __init__(self, texto):
            self._texto = texto

        def renderizar(self):
            return self._texto


    class DecoradorDeConteudo(Conteudo):
        """Decorador base: mantem a referencia ao componente envolvido."""

        def __init__(self, conteudo):
            if not isinstance(conteudo, Conteudo):
                raise TypeError("Um decorador so envolve objetos do tipo Conteudo.")
            self._conteudo = conteudo

        def renderizar(self):
            return self._conteudo.renderizar()


    class ComCensura(DecoradorDeConteudo):
        """Substitui as palavras proibidas por asteriscos."""

        PALAVRAS_PROIBIDAS = ("idiota", "burro", "lixo")

        def renderizar(self):
            texto = super().renderizar()
            for palavra in self.PALAVRAS_PROIBIDAS:
                texto = re.sub(
                    palavra,
                    "*" * len(palavra),
                    texto,
                    flags=re.IGNORECASE,
                )
            return texto


    class ComDestaque(DecoradorDeConteudo):
        """Marca o texto como aviso de moderacao."""

        def renderizar(self):
            return f">>> {super().renderizar()} <<<"


    class ComAssinatura(DecoradorDeConteudo):
        """Acrescenta a assinatura do autor ao final do texto."""

        def __init__(self, conteudo, usuario):
            super().__init__(conteudo)
            self._usuario = usuario

        def renderizar(self):
            return f"{super().renderizar()}\n-- {self._usuario.apelido} ({self._usuario.papel})"


    DECORADORES = {
        "censura": ComCensura,
        "destaque": ComDestaque,
        "assinatura": ComAssinatura,
    }
```

Arquivo: `forum/estrutural/facade.py`

```
    """Padrao FACADE (estrutural).

    Intencao
        Fornecer uma interface unificada para um conjunto de interfaces de um
        subsistema. O Facade define uma interface de nivel mais alto que torna
        o subsistema mais facil de usar.

    Aplicacao no forum
        Abrir uma discussao exige coordenar seis participantes: o repositorio
        unico (Singleton), o construtor passo a passo (Builder), o criador do
        produto (Factory Method), a arvore da discussao (Composite), os
        inscritos (Observer) e o estado inicial (State). Quem chama precisa de
        um unico metodo.

        No projeto cliente/servidor esta fachada e a camada de servico do
        servidor Flask: cada endpoint chama um metodo daqui e traduz o
        resultado em JSON. Nenhuma rota conhece as classes dos padroes, e o
        dominio nao importa nada do Flask.

    Erros
        A fachada nao imprime mensagens: em caso de regra violada ela levanta
        ErroDeForum com o status HTTP adequado, e a camada de API traduz a
        excecao em resposta.

    Participantes
        Facade     -> ForumFacade
        Subsistema -> Forum, ConstrutorDeTopico, criadores, Topico/Mensagem/Anexo,
                      observadores, estados e estrategias de ordenacao
    """
```

```python
from ..comportamental.observer import NotificadorDeUsuario, RegistroDeAtividade
from ..comportamental.strategy import ESTRATEGIAS, ListagemDeTopicos, PorMaisRecentes
from ..criacional.builder import ConstrutorDeTopico
from ..criacional.factory_method import CriadorDeAnexo, CriadorDeMensagem
from ..criacional.singleton import Forum
from ..erros import ErroDeForum
from ..modelos import Usuario
from ..rastro import rastrear
from .composite import Mensagem, Topico
from .decorator import DECORADORES, ComAssinatura, ConteudoSimples


class ForumFacade:
    """Interface unica de alto nivel para todo o subsistema do forum."""

    ACOES_DE_ESTADO = ("abrir", "fechar", "arquivar")

    def __init__(self):
        self._forum = Forum()
        self._construtor = ConstrutorDeTopico()
        self._listagem = ListagemDeTopicos(PorMaisRecentes())
        self._registro = RegistroDeAtividade()

    @property
    def forum(self):
        return self._forum

    @property
    def registro(self):
        return self._registro

    # --- usuarios ------------------------------------------------------------
    def cadastrar_usuario(self, apelido, email, papel="membro"):
        apelido = (apelido or "").strip()
        email = (email or "").strip()
        if not apelido:
            raise ErroDeForum("O apelido nao pode ser vazio.", 400)
        if not email:
            raise ErroDeForum("O e-mail nao pode ser vazio.", 400)
        if self._forum.obter_usuario(apelido) is not None:
            raise ErroDeForum(f'Ja existe um usuario com o apelido "{apelido}".', 409)
        try:
            usuario = Usuario(apelido, email, papel)
        except ValueError as erro:
            raise ErroDeForum(str(erro), 400) from erro

        self._forum.registrar_usuario(usuario)
        rastrear("Facade", f'Usuario "{usuario.apelido}" gravado no repositorio unico')
        return usuario

    def listar_usuarios(self):
        return list(self._forum.usuarios)

    def obter_usuario(self, apelido):
        usuario = self._forum.obter_usuario((apelido or "").strip())
        if usuario is None:
            raise ErroDeForum(f'Usuario "{apelido}" nao cadastrado.', 404)
        return usuario

    def caixa_de_entrada(self, apelido):
        return list(self.obter_usuario(apelido).caixa_entrada)

    # --- topicos -------------------------------------------------------------
    def abrir_discussao(self, apelido_autor, titulo, conteudo="", tags=None, fixado=False):
        autor = self.obter_usuario(apelido_autor)
        if fixado and not autor.e_moderador:
            raise ErroDeForum("Somente moderadores podem fixar um topico.", 403)

        rastrear("Facade", "Coordenando Builder + Factory Method + Singleton + Observer")
        try:
            topico = (
                self._construtor.com_identificador(self._forum.proximo_identificador())
                .com_titulo(titulo)
                .com_autor(autor)
                .com_conteudo(conteudo)
                .com_tags(tags)
                .fixado(fixado)
                .construir()
            )
        except ValueError as erro:
            raise ErroDeForum(str(erro), 400) from erro

        self._forum.adicionar_topico(topico)
        topico.inscrever(NotificadorDeUsuario(autor))
        topico.inscrever(self._registro)
        rastrear("Facade", f"Topico guardado em {topico.caminho()} e observadores inscritos")
        return topico

    def obter_topico(self, identificador):
        topico = self._forum.obter_topico(identificador)
        if topico is None:
            raise ErroDeForum(f"Topico #{identificador} nao encontrado.", 404)
        rastrear("Composite", f"Percorrendo a arvore a partir de {topico.caminho()}")
        return topico

    def listar_topicos(self, chave_estrategia=None):
        if chave_estrategia:
            if chave_estrategia not in ESTRATEGIAS:
                validas = ", ".join(ESTRATEGIAS)
```

```
                raise ErroDeForum(
                    f'Ordenacao desconhecida: "{chave_estrategia}". Use uma de: {validas}.', 400
                )
            self._listagem.definir_estrategia(ESTRATEGIAS[chave_estrategia]())
        return self._listagem.listar(self._forum.topicos)

    def responder(self, identificador_pai, apelido_autor, texto):
        autor = self.obter_usuario(apelido_autor)
        texto = (texto or "").strip()
        if not texto:
            raise ErroDeForum("A resposta nao pode ser vazia.", 400)

        alvo = self._localizar(identificador_pai)
        if not isinstance(alvo, (Topico, Mensagem)):
            raise ErroDeForum("Anexos sao folhas da arvore e nao recebem respostas.", 400)

        topico = alvo.raiz()
        rastrear("Facade", f"Consultando o estado do topico #{topico.identificador}")
        if not topico.aceita_resposta():
            raise ErroDeForum(
                f"O topico esta {topico.estado.nome} e nao aceita novas respostas.", 409
            )

        mensagem = CriadorDeMensagem().publicar(
            self._forum.proximo_identificador(), autor, conteudo=texto
        )
        alvo.adicionar(mensagem)
        rastrear("Composite", f"Mensagem inserida em {mensagem.caminho()}")
        topico.notificar("nova-resposta", f'{autor.apelido} respondeu em "{topico.titulo}"')
        return mensagem

    def anexar(self, identificador_mensagem, apelido_autor, nome_arquivo, tamanho_kb=0):
        autor = self.obter_usuario(apelido_autor)
        nome_arquivo = (nome_arquivo or "").strip()
        if not nome_arquivo:
            raise ErroDeForum("O nome do arquivo nao pode ser vazio.", 400)
        try:
            tamanho_kb = int(tamanho_kb)
        except (TypeError, ValueError) as erro:
            raise ErroDeForum("O tamanho do anexo deve ser um numero inteiro.", 400) from erro
        if tamanho_kb < 0:
            raise ErroDeForum("O tamanho do anexo nao pode ser negativo.", 400)

        alvo = self._localizar(identificador_mensagem)
        try:
            anexo = CriadorDeAnexo().publicar(
                self._forum.proximo_identificador(),
                autor,
                nome_arquivo=nome_arquivo,
                tamanho_kb=tamanho_kb,
            )
            alvo.adicionar(anexo)
        except TypeError as erro:
            raise ErroDeForum(str(erro), 400) from erro
        rastrear("Composite", f"Folha inserida em {anexo.caminho()}")
        return anexo

    # --- observer ------------------------------------------------------------
    def inscrever(self, identificador_topico, apelido):
        topico = self._forum.obter_topico(identificador_topico)
        if topico is None:
            raise ErroDeForum(f"Topico #{identificador_topico} nao encontrado.", 404)
        usuario = self.obter_usuario(apelido)
        if not topico.inscrever(NotificadorDeUsuario(usuario)):
            raise ErroDeForum(f'"{apelido}" ja esta inscrito neste topico.', 409)
        rastrear(
            "Observer",
            f'"{apelido}" inscrito. Total de observadores: {len(topico.observadores)}',
        )
        return topico

    # --- state ---------------------------------------------------------------
    def alterar_estado(self, identificador_topico, acao):
        topico = self._forum.obter_topico(identificador_topico)
        if topico is None:
            raise ErroDeForum(f"Topico #{identificador_topico} nao encontrado.", 404)
        if acao not in self.ACOES_DE_ESTADO:
            validas = ", ".join(self.ACOES_DE_ESTADO)
            raise ErroDeForum(f'Acao invalida: "{acao}". Use uma de: {validas}.', 400)

        anterior = topico.estado.nome
        rastrear("Facade", f'Pedindo "{acao}" ao estado atual ({anterior})')
        if not getattr(topico, acao)():
            raise ErroDeForum(
                f'Nao e possivel "{acao}" um topico {anterior}.', 409
            )
        return topico

    # --- votos ---------------------------------------------------------------
    def votar(self, identificador_mensagem):
        item = self._localizar(identificador_mensagem)
        if not isinstance(item, Mensagem):
            raise ErroDeForum("Somente mensagens podem receber votos.", 400)
        total = item.votar()
        rastrear(None, f"Voto registrado em {item.caminho()} (total: {total})")
        return item

    # --- decorator -----------------------------------------------------------
    def formatar(self, texto, filtros, apelido_autor=None):
        if texto is None or not str(texto).strip():
```

```
                raise ErroDeForum("O texto a formatar nao pode ser vazio.", 400)

            conteudo = ConteudoSimples(texto)
            aplicados = []
            for filtro in filtros or []:
                if filtro not in DECORADORES:
                    validos = ", ".join(DECORADORES)
                    raise ErroDeForum(
                        f'Filtro desconhecido: "{filtro}". Use um de: {validos}.', 400
                    )
                if filtro == "assinatura":
                    conteudo = ComAssinatura(conteudo, self.obter_usuario(apelido_autor))
                else:
                    conteudo = DECORADORES[filtro](conteudo)
                aplicados.append(filtro)

            rastrear(
                "Decorator",
                "Pilha aplicada (de dentro para fora): "
                + (" -> ".join(aplicados) if aplicados else "(nenhum filtro)"),
            )
            return conteudo.renderizar()

        # --- consultas ------------------------------------------------------
        def buscar(self, termo):
            termo = (termo or "").strip()
            if not termo:
                raise ErroDeForum("O termo de busca nao pode ser vazio.", 400)
            rastrear("Facade", f'Buscando "{termo}" em todas as arvores de discussao')
            encontrados = self._forum.buscar_por_texto(termo)
            rastrear(None, f"{len(encontrados)} item(ns) encontrado(s)")
            return encontrados

        def estatisticas(self):
            dados = self._forum.estatisticas()
            dados["eventos_registrados"] = len(self._registro.eventos)
            dados["instancia_unica"] = Forum() is self._forum
            rastrear(
                "Singleton",
                f"Forum() devolve o mesmo objeto? {dados['instancia_unica']}",
            )
            return dados

        # --- apoio ----------------------------------------------------------
        def _localizar(self, identificador):
            item = self._forum.localizar(identificador)
            if item is None:
                raise ErroDeForum(f"Nenhum item encontrado com o identificador #{identificador}.", 404)
            return item
```

Arquivo: `forum/comportamental/observer.py`

```
"""Padrao OBSERVER (comportamental).

Intencao
    Definir uma dependencia um-para-muitos entre objetos, de modo que
    quando um objeto muda de estado todos os seus dependentes sao
    notificados e atualizados automaticamente.

Aplicacao no forum
    Um topico (Assunto) mantem a lista de usuarios inscritos. Quando uma
    nova resposta e publicada, todos os inscritos (Observadores) recebem
    a notificacao sem que o topico conheca a classe concreta de cada um.

Participantes
    Assunto (Subject)               -> Assunto
    Observador (Observer)           -> Observador
    Observador concreto             -> NotificadorDeUsuario, RegistroDeAtividade
"""

from abc import ABC, abstractmethod

from ..rastro import rastrear


class Observador(ABC):
    """Interface comum a todos os observadores."""

    @abstractmethod
    def atualizar(self, assunto, evento, dados):
        """Reage a um evento emitido pelo assunto observado."""


class Assunto:
    """Mantem a lista de observadores e os notifica."""

    def __init__(self):
        self._observadores = []

    @property
    def observadores(self):
        return tuple(self._observadores)

    def inscrever(self, observador):
        if observador in self._observadores:
            return False
        self._observadores.append(observador)
        return True
```

```
    def desinscrever(self, observador):
        if observador not in self._observadores:
            return False
        self._observadores.remove(observador)
        return True

    def notificar(self, evento, dados):
        total = len(self._observadores)
        rastrear("Observer", f'Notificando {total} inscrito(s) sobre o evento "{evento}"')
        if total == 0:
            rastrear(None, "  -> nenhum inscrito para notificar")
            return
        for observador in list(self._observadores):
            observador.atualizar(self, evento, dados)


class NotificadorDeUsuario(Observador):
    """Entrega a notificacao na caixa de entrada de um usuario."""

    def __init__(self, usuario):
        self.usuario = usuario

    def atualizar(self, assunto, evento, dados):
        self.usuario.caixa_entrada.append(f"[{evento}] {dados}")
        rastrear(None, f"  -> caixa de entrada de {self.usuario.apelido}: {dados}")

    def __eq__(self, outro):
        return (
            isinstance(outro, NotificadorDeUsuario)
            and outro.usuario.apelido == self.usuario.apelido
        )

    def __hash__(self):
        return hash(("NotificadorDeUsuario", self.usuario.apelido))


class RegistroDeAtividade(Observador):
    """Observador de auditoria: guarda o historico de eventos do forum."""

    def __init__(self):
        self.eventos = []

    def atualizar(self, assunto, evento, dados):
        self.eventos.append((evento, dados))
        rastrear(None, f"  -> registro de atividade: {evento} gravado (total: {len(self.eventos)})")

    def __eq__(self, outro):
        return isinstance(outro, RegistroDeAtividade)

    def __hash__(self):
        return hash("RegistroDeAtividade")
```

Arquivo: `forum/comportamental/strategy.py`

```
"""Padrao STRATEGY (comportamental).

Intencao
    Definir uma familia de algoritmos, encapsular cada um deles e
    torna-los intercambiaveis. O Strategy permite que o algoritmo varie
    independentemente dos clientes que o utilizam.

Aplicacao no forum
    A pagina inicial lista os topicos, mas a ordem depende da preferencia
    do usuario: mais recentes, mais respondidos ou mais votados. O
    contexto ListagemDeTopicos nao conhece os algoritmos; ele apenas
    delega a estrategia configurada, que pode ser trocada em execucao.

Participantes
    Contexto (Context)        -> ListagemDeTopicos
    Estrategia (Strategy)     -> EstrategiaDeOrdenacao
    Estrategias concretas     -> PorMaisRecentes, PorMaisRespondidos, PorMaisVotados
"""

from abc import ABC, abstractmethod

from ..rastro import rastrear


class EstrategiaDeOrdenacao(ABC):
    """Interface comum aos algoritmos de ordenacao."""

    nome = "indefinida"

    @abstractmethod
    def ordenar(self, topicos):
        """Devolve uma nova lista de topicos na ordem do algoritmo."""


class PorMaisRecentes(EstrategiaDeOrdenacao):
    """Identificadores maiores foram criados por ultimo."""

    nome = "mais recentes"

    def ordenar(self, topicos):
        return sorted(topicos, key=lambda t: t.identificador, reverse=True)
```

```
class PorMaisRespondidos(EstrategiaDeOrdenacao):
    nome = "mais respondidos"

    def ordenar(self, topicos):
        return sorted(topicos, key=lambda t: (-t.total_respostas(), t.identificador))


class PorMaisVotados(EstrategiaDeOrdenacao):
    nome = "mais votados"

    def ordenar(self, topicos):
        return sorted(topicos, key=lambda t: (-t.total_votos(), t.identificador))


ESTRATEGIAS = {
    "recentes": PorMaisRecentes,
    "respondidos": PorMaisRespondidos,
    "votados": PorMaisVotados,
}


class ListagemDeTopicos:
    """Contexto: usa a estrategia configurada sem conhecer o algoritmo."""

    def __init__(self, estrategia=None):
        self._estrategia = estrategia or PorMaisRecentes()

    @property
    def estrategia(self):
        return self._estrategia

    def definir_estrategia(self, estrategia):
        if type(estrategia) is type(self._estrategia):
            rastrear("Strategy", f"Estrategia mantida: {estrategia.nome}")
            return
        anterior = self._estrategia.nome
        self._estrategia = estrategia
        rastrear("Strategy", f"Estrategia trocada: {anterior} -> {estrategia.nome}")

    def listar(self, topicos):
        rastrear("Strategy", f"Ordenando os topicos por: {self._estrategia.nome}")
        return self._estrategia.ordenar(list(topicos))
```

Arquivo: `forum/comportamental/state.py`

```
"""Padrao STATE (comportamental).

Intencao
    Permitir que um objeto altere seu comportamento quando seu estado
    interno muda; o objeto parece mudar de classe.

Aplicacao no forum
    Um topico pode estar Aberto, Fechado ou Arquivado. A permissao para
    receber respostas e as transicoes validas ficam dentro de cada classe
    de estado, eliminando os encadeamentos de "if" espalhados pelo codigo.

Transicoes validas
    Aberto    -> Fechado, Arquivado
    Fechado   -> Aberto, Arquivado
    Arquivado -> (nenhuma: estado terminal)

Participantes
    Contexto (Context)   -> Topico (forum.estrutural.composite)
    Estado (State)       -> EstadoTopico
    Estados concretos    -> Aberto, Fechado, Arquivado
"""

from abc import ABC, abstractmethod

from ..rastro import rastrear


class EstadoTopico(ABC):
    """Interface comum aos estados de um topico."""

    nome = "indefinido"

    @abstractmethod
    def pode_receber_resposta(self):
        """Informa se o topico aceita novas respostas neste estado."""

    def abrir(self, topico):
        return self._recusar(topico, "abrir")

    def fechar(self, topico):
        return self._recusar(topico, "fechar")

    def arquivar(self, topico):
        return self._recusar(topico, "arquivar")

    def _trocar(self, topico, novo_estado):
        rastrear("State", f"Transicao: {self.nome} -> {novo_estado.nome}")
        topico.definir_estado(novo_estado)
        return True

    def _recusar(self, topico, acao):
        rastrear("State", f'Transicao recusada: nao e possivel "{acao}" um topico {self.nome}')
        return False
```

```
        def __str__(self):
            return self.nome


    class Aberto(EstadoTopico):
        nome = "Aberto"

        def pode_receber_resposta(self):
            return True

        def abrir(self, topico):
            rastrear("State", "O topico ja esta Aberto")
            return False

        def fechar(self, topico):
            return self._trocar(topico, Fechado())

        def arquivar(self, topico):
            return self._trocar(topico, Arquivado())


    class Fechado(EstadoTopico):
        nome = "Fechado"

        def pode_receber_resposta(self):
            return False

        def abrir(self, topico):
            return self._trocar(topico, Aberto())

        def fechar(self, topico):
            rastrear("State", "O topico ja esta Fechado")
            return False

        def arquivar(self, topico):
            return self._trocar(topico, Arquivado())


    class Arquivado(EstadoTopico):
        """Estado terminal: nao permite nenhuma transicao de saida."""

        nome = "Arquivado"

        def pode_receber_resposta(self):
            return False

        def arquivar(self, topico):
            rastrear("State", "O topico ja esta Arquivado")
            return False
```

7.2 Servidor

Arquivo: `servidor/app.py`

```
    """Servidor Flask do forum de discussao.

    Cada endpoint e uma casca fina: valida a entrada, chama um metodo da
    fachada do dominio (forum.estrutural.facade.ForumFacade) e devolve JSON.
    Nenhuma rota conhece as classes dos padroes de projeto, e o dominio nao
    importa nada do Flask.

    Toda resposta tem o mesmo envelope:

        {
          "dados":  <resultado da operacao>,
          "rastro": ["[Builder] ...", "[Factory Method] ...", ...]
        }

    O campo "rastro" traz os padroes acionados naquela requisicao. O cliente
    grafico mostra essas linhas em um painel lateral, o que torna o
    funcionamento dos padroes visivel durante a demonstracao.

    Execucao:
        python -m servidor
    """

    from flask import Flask, jsonify, request

    from forum import rastro
    from forum.erros import ErroDeForum
    from forum.estrutural.facade import ForumFacade

    ENDPOINTS = [
        ("GET", "/api/saude", "Estado do servidor"),
        ("GET", "/api/usuarios", "Lista os usuarios cadastrados"),
        ("POST", "/api/usuarios", "Cadastra um usuario"),
        ("GET", "/api/usuarios/<apelido>/notificacoes", "Caixa de entrada do usuario (Observer)"),
        ("GET", "/api/topicos", "Lista topicos; ?ordenacao=recentes|respondidos|votados (Strategy)"),
        ("POST", "/api/topicos", "Abre um topico (Builder + Factory Method)"),
        ("GET", "/api/topicos/<id>", "Arvore completa da discussao (Composite)"),
        ("POST", "/api/topicos/<id>/inscricoes", "Inscreve um usuario no topico (Observer)"),
        ("PUT", "/api/topicos/<id>/estado", "Abre, fecha ou arquiva o topico (State)"),
        ("POST", "/api/itens/<id>/respostas", "Responde a um topico ou a uma mensagem"),
        ("POST", "/api/mensagens/<id>/anexos", "Anexa um arquivo a uma mensagem (Composite)"),
        ("POST", "/api/mensagens/<id>/votos", "Registra um voto na mensagem"),
```

```
        ("POST", "/api/conteudo/formatado", "Aplica a pilha de filtros ao texto (Decorator)"),
        ("GET", "/api/busca", "Busca por texto em todas as arvores; ?termo="),
        ("GET", "/api/estatisticas", "Numeros do forum (Singleton)"),
    ]


    def resumo_de_topico(topico):
        """Versao curta de um topico, usada nas listagens."""
        return {
            "identificador": topico.identificador,
            "titulo": topico.titulo,
            "autor": topico.autor.apelido,
            "estado": topico.estado.nome,
            "fixado": topico.fixado,
            "tags": list(topico.tags),
            "total_respostas": topico.total_respostas(),
            "total_votos": topico.total_votos(),
            "aceita_resposta": topico.aceita_resposta(),
            "rotulo": topico.rotulo(),
            "caminho": topico.caminho(),
        }


    def resumo_de_item(item):
        """Versao curta de qualquer no da arvore, usada na busca."""
        return {
            "identificador": item.identificador,
            "tipo": item.tipo,
            "caminho": item.caminho(),
            "rotulo": item.rotulo(),
            "autor": item.autor.apelido,
            "topico": item.raiz().identificador,
        }


    def criar_app(fachada=None):
        """Monta a aplicacao Flask.

        Receber a fachada por parametro mantem o servidor testavel: os testes
        passam uma fachada sobre um forum limpo.
        """
        app = Flask(__name__)
        app.json.ensure_ascii = False
        servico = fachada or ForumFacade()
        app.extensions["fachada"] = servico

        # --- envelope de resposta ------------------------------------------------
        def responder(dados, status=200):
            linhas = rastro.coletar()
            for linha in linhas:
                app.logger.info(linha)
            return jsonify({"dados": dados, "rastro": linhas}), status

        def corpo():
            return request.get_json(silent=True) or {}

        @app.before_request
        def preparar_rastro():
            rastro.iniciar()

        @app.errorhandler(ErroDeForum)
        def tratar_erro_de_dominio(erro):
            return jsonify(
                {"erro": erro.mensagem, "rastro": rastro.coletar()}
            ), erro.codigo

        @app.errorhandler(404)
        def tratar_rota_inexistente(_erro):
            return jsonify({"erro": "Endpoint nao encontrado."}), 404

        @app.errorhandler(405)
        def tratar_metodo_invalido(_erro):
            return jsonify({"erro": "Metodo nao permitido para este endpoint."}), 405

        # --- raiz e saude --------------------------------------------------------
        @app.get("/")
        def indice():
            return jsonify(
                {
                    "aplicacao": "Forum de Discussao - projeto de Design Patterns",
                    "endpoints": [
                        {"metodo": m, "rota": r, "descricao": d} for m, r, d in ENDPOINTS
                    ],
                }
            )

        @app.get("/api/saude")
        def saude():
            return responder({"status": "ok"})

        # --- usuarios ------------------------------------------------------------
        @app.get("/api/usuarios")
        def listar_usuarios():
            usuarios = [
                {"apelido": u.apelido, "email": u.email, "papel": u.papel}
                for u in servico.listar_usuarios()
            ]
            return responder(usuarios)

        @app.post("/api/usuarios")
```

```
            def cadastrar_usuario():
                dados = corpo()
                usuario = servico.cadastrar_usuario(
                    dados.get("apelido"), dados.get("email"), dados.get("papel", "membro")
                )
                return responder(
                    {"apelido": usuario.apelido, "email": usuario.email, "papel": usuario.papel},
                    201,
                )

            @app.get("/api/usuarios/<apelido>/notificacoes")
            def notificacoes(apelido):
                return responder(servico.caixa_de_entrada(apelido))

            # --- topicos ------------------------------------------------------------
            @app.get("/api/topicos")
            def listar_topicos():
                ordenacao = request.args.get("ordenacao")
                topicos = servico.listar_topicos(ordenacao)
                return responder([resumo_de_topico(t) for t in topicos])

            @app.post("/api/topicos")
            def abrir_topico():
                dados = corpo()
                topico = servico.abrir_discussao(
                    dados.get("autor"),
                    dados.get("titulo"),
                    dados.get("conteudo", ""),
                    dados.get("tags"),
                    bool(dados.get("fixado", False)),
                )
                return responder(topico.para_dicionario(), 201)

            @app.get("/api/topicos/<int:identificador>")
            def obter_topico(identificador):
                topico = servico.obter_topico(identificador)
                return responder(topico.para_dicionario())

            @app.post("/api/topicos/<int:identificador>/inscricoes")
            def inscrever(identificador):
                topico = servico.inscrever(identificador, corpo().get("apelido"))
                return responder(resumo_de_topico(topico), 201)

            @app.put("/api/topicos/<int:identificador>/estado")
            def alterar_estado(identificador):
                topico = servico.alterar_estado(identificador, corpo().get("acao"))
                return responder(resumo_de_topico(topico))

            # --- mensagens e anexos -------------------------------------------------
            @app.post("/api/itens/<int:identificador>/respostas")
            def responder_item(identificador):
                dados = corpo()
                mensagem = servico.responder(identificador, dados.get("autor"), dados.get("conteudo"))
                return responder(mensagem.para_dicionario(), 201)

            @app.post("/api/mensagens/<int:identificador>/anexos")
            def anexar(identificador):
                dados = corpo()
                anexo = servico.anexar(
                    identificador,
                    dados.get("autor"),
                    dados.get("nome_arquivo"),
                    dados.get("tamanho_kb", 0),
                )
                return responder(anexo.para_dicionario(), 201)

            @app.post("/api/mensagens/<int:identificador>/votos")
            def votar(identificador):
                mensagem = servico.votar(identificador)
                return responder(mensagem.para_dicionario())

            # --- conteudo, busca e estatisticas -------------------------------------
            @app.post("/api/conteudo/formatado")
            def formatar():
                dados = corpo()
                texto = servico.formatar(
                    dados.get("texto"), dados.get("filtros") or [], dados.get("autor")
                )
                return responder({"texto": texto})

            @app.get("/api/busca")
            def buscar():
                encontrados = servico.buscar(request.args.get("termo"))
                return responder([resumo_de_item(item) for item in encontrados])

            @app.get("/api/estatisticas")
            def estatisticas():
                return responder(servico.estatisticas())

            return app


        if __name__ == "__main__":
            criar_app().run(host="127.0.0.1", port=5000, debug=False)
```

Arquivo: `servidor/__main__.py`

```
        """Ponto de entrada do servidor: python -m servidor"""
```

```
from .app import criar_app

ENDERECO = "127.0.0.1"
PORTA = 5000

if __name__ == "__main__":
    print(f"Servidor do forum em http://{ENDERECO}:{PORTA}")
    print("Pressione Ctrl+C para encerrar.")
    criar_app().run(host=ENDERECO, port=PORTA, debug=False)
```

7.3 Cliente

Arquivo: `cliente/api.py`

```
"""Acesso do cliente aos endpoints do servidor.

ApiForum e um proxy remoto: expoe os mesmos metodos da fachada do dominio,
mas em vez de executar a operacao, empacota a chamada em uma requisicao
HTTP. A janela grafica conversa so com esta classe e nunca monta uma URL
ou um cabecalho por conta propria.

Toda chamada devolve uma Resposta com os dados e o rastro de padroes que o
servidor acionou, que a janela exibe no painel lateral.
"""

from dataclasses import dataclass, field

import requests

ENDERECO_PADRAO = "http://127.0.0.1:5000"
TEMPO_LIMITE = 10


@dataclass
class Resposta:
    """Resultado de uma chamada bem-sucedida."""

    dados: object = None
    rastro: list = field(default_factory=list)


class ErroDaApi(Exception):
    """Erro devolvido pelo servidor ou falha de comunicacao."""

    def __init__(self, mensagem, codigo=None, rastro=None):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.codigo = codigo
        self.rastro = rastro or []


class ApiForum:
    def __init__(self, endereco=ENDERECO_PADRAO, sessao=None):
        self.endereco = endereco.rstrip("/")
        self._sessao = sessao or requests.Session()

    # --- transporte ---------------------------------------------------------
    def _chamar(self, metodo, rota, **kwargs):
        url = f"{self.endereco}{rota}"
        try:
            resposta = self._sessao.request(metodo, url, timeout=TEMPO_LIMITE, **kwargs)
        except requests.RequestException as erro:
            raise ErroDaApi(
                f"Nao foi possivel falar com o servidor em {self.endereco}. "
                "Verifique se ele esta em execucao (python -m servidor)."
            ) from erro

        try:
            corpo = resposta.json()
        except ValueError as erro:
            raise ErroDaApi(
                f"O servidor respondeu algo que nao e JSON (HTTP {resposta.status_code}).",
                resposta.status_code,
            ) from erro

        if resposta.status_code >= 400:
            raise ErroDaApi(
                corpo.get("erro", "Erro desconhecido no servidor."),
                resposta.status_code,
                corpo.get("rastro"),
            )
        return Resposta(corpo.get("dados"), corpo.get("rastro") or [])

    # --- operacoes ----------------------------------------------------------
    def saude(self):
        return self._chamar("GET", "/api/saude")

    def listar_usuarios(self):
        return self._chamar("GET", "/api/usuarios")

    def cadastrar_usuario(self, apelido, email, papel="membro"):
        return self._chamar(
            "POST", "/api/usuarios",
            json={"apelido": apelido, "email": email, "papel": papel},
        )
```

```
    def notificacoes(self, apelido):
        return self._chamar("GET", f"/api/usuarios/{apelido}/notificacoes")

    def listar_topicos(self, ordenacao=None):
        parametros = {"ordenacao": ordenacao} if ordenacao else None
        return self._chamar("GET", "/api/topicos", params=parametros)

    def abrir_topico(self, autor, titulo, conteudo, tags=None, fixado=False):
        return self._chamar(
            "POST", "/api/topicos",
            json={
                "autor": autor,
                "titulo": titulo,
                "conteudo": conteudo,
                "tags": tags or [],
                "fixado": fixado,
            },
        )

    def obter_topico(self, identificador):
        return self._chamar("GET", f"/api/topicos/{identificador}")

    def inscrever(self, identificador, apelido):
        return self._chamar(
            "POST", f"/api/topicos/{identificador}/inscricoes", json={"apelido": apelido}
        )

    def alterar_estado(self, identificador, acao):
        return self._chamar(
            "PUT", f"/api/topicos/{identificador}/estado", json={"acao": acao}
        )

    def responder(self, identificador, autor, conteudo):
        return self._chamar(
            "POST", f"/api/itens/{identificador}/respostas",
            json={"autor": autor, "conteudo": conteudo},
        )

    def anexar(self, identificador, autor, nome_arquivo, tamanho_kb):
        return self._chamar(
            "POST", f"/api/mensagens/{identificador}/anexos",
            json={"autor": autor, "nome_arquivo": nome_arquivo, "tamanho_kb": tamanho_kb},
        )

    def votar(self, identificador):
        return self._chamar("POST", f"/api/mensagens/{identificador}/votos")

    def formatar(self, texto, filtros, autor=None):
        return self._chamar(
            "POST", "/api/conteudo/formatado",
            json={"texto": texto, "filtros": filtros, "autor": autor},
        )

    def buscar(self, termo):
        return self._chamar("GET", "/api/busca", params={"termo": termo})

    def estatisticas(self):
        return self._chamar("GET", "/api/estatisticas")
```

Arquivo: `cliente/dialogos.py`

```
"""Caixas de dialogo do cliente grafico."""

from PySide6.QtWidgets import (
    QCheckBox,
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QFormLayout,
    QGroupBox,
    QLabel,
    QLineEdit,
    QPlainTextEdit,
    QSpinBox,
    QVBoxLayout,
)


class _DialogoBase(QDialog):
    """Dialogo com os botoes OK/Cancelar ja ligados."""

    def __init__(self, titulo, pai=None):
        super().__init__(pai)
        self.setWindowTitle(titulo)
        self.setMinimumWidth(420)
        self._layout = QVBoxLayout(self)

    def _adicionar_botoes(self):
        botoes = QDialogButtonBox(
            QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel
        )
        botoes.accepted.connect(self.accept)
        botoes.rejected.connect(self.reject)
        self._layout.addWidget(botoes)
        return botoes
```

```
        class DialogoUsuario(_DialogoBase):
            """Cadastro de um novo participante."""

            def __init__(self, pai=None):
                super().__init__("Cadastrar usuario", pai)
                formulario = QFormLayout()
                self.apelido = QLineEdit()
                self.email = QLineEdit()
                self.papel = QComboBox()
                self.papel.addItems(["membro", "moderador"])
                formulario.addRow("Apelido:", self.apelido)
                formulario.addRow("E-mail:", self.email)
                formulario.addRow("Papel:", self.papel)
                self._layout.addLayout(formulario)
                self._adicionar_botoes()

            def valores(self):
                return {
                    "apelido": self.apelido.text().strip(),
                    "email": self.email.text().strip(),
                    "papel": self.papel.currentText(),
                }


        class DialogoNovoTopico(_DialogoBase):
            """Reune as partes que o Builder monta no servidor."""

            def __init__(self, pai=None, pode_fixar=False):
                super().__init__("Abrir novo topico", pai)
                formulario = QFormLayout()
                self.titulo = QLineEdit()
                self.conteudo = QPlainTextEdit()
                self.conteudo.setMinimumHeight(110)
                self.tags = QLineEdit()
                self.tags.setPlaceholderText("separadas por virgula")
                self.fixado = QCheckBox("Fixar no topo da lista")
                self.fixado.setEnabled(pode_fixar)
                if not pode_fixar:
                    self.fixado.setToolTip("Somente moderadores podem fixar um topico.")
                formulario.addRow("Titulo:", self.titulo)
                formulario.addRow("Texto de abertura:", self.conteudo)
                formulario.addRow("Tags:", self.tags)
                formulario.addRow("", self.fixado)
                self._layout.addLayout(formulario)
                self._adicionar_botoes()

            def valores(self):
                return {
                    "titulo": self.titulo.text().strip(),
                    "conteudo": self.conteudo.toPlainText().strip(),
                    "tags": [t.strip() for t in self.tags.text().split(",") if t.strip()],
                    "fixado": self.fixado.isChecked(),
                }


        class DialogoResposta(_DialogoBase):
            def __init__(self, destino, pai=None):
                super().__init__("Responder", pai)
                self._layout.addWidget(QLabel(f"Respondendo a: {destino}"))
                self.texto = QPlainTextEdit()
                self.texto.setMinimumHeight(120)
                self._layout.addWidget(self.texto)
                self._adicionar_botoes()

            def valores(self):
                return {"conteudo": self.texto.toPlainText().strip()}


        class DialogoAnexo(_DialogoBase):
            def __init__(self, destino, pai=None):
                super().__init__("Anexar arquivo", pai)
                self._layout.addWidget(QLabel(f"Anexando em: {destino}"))
                formulario = QFormLayout()
                self.nome = QLineEdit()
                self.nome.setPlaceholderText("exemplo.py")
                self.tamanho = QSpinBox()
                self.tamanho.setRange(0, 1_000_000)
                self.tamanho.setSuffix(" KB")
                formulario.addRow("Nome do arquivo:", self.nome)
                formulario.addRow("Tamanho:", self.tamanho)
                self._layout.addLayout(formulario)
                self._adicionar_botoes()

            def valores(self):
                return {"nome_arquivo": self.nome.text().strip(), "tamanho_kb": self.tamanho.value()}


        class DialogoFormatar(_DialogoBase):
            """Monta a pilha de decoradores e mostra o texto renderizado."""

            def __init__(self, pai=None):
                super().__init__("Formatar conteudo (Decorator)", pai)
                self._layout.addWidget(QLabel("Texto original:"))
                self.texto = QPlainTextEdit()
                self.texto.setMinimumHeight(90)
                self._layout.addWidget(self.texto)

                grupo = QGroupBox("Filtros, aplicados de cima para baixo")
                caixa = QVBoxLayout(grupo)
                self.censura = QCheckBox("Censura de palavras proibidas")
```

```
            self.destaque = QCheckBox("Destaque de moderacao")
            self.assinatura = QCheckBox("Assinatura do usuario atual")
            for caixinha in (self.censura, self.destaque, self.assinatura):
                caixa.addWidget(caixinha)
            self._layout.addWidget(grupo)

            self._layout.addWidget(QLabel("Resultado:"))
            self.resultado = QPlainTextEdit()
            self.resultado.setReadOnly(True)
            self.resultado.setMinimumHeight(80)
            self._layout.addWidget(self.resultado)

            botoes = self._adicionar_botoes()
            self.botao_aplicar = botoes.addButton(
                "Aplicar", QDialogButtonBox.ButtonRole.ActionRole
            )

        def filtros(self):
            escolhidos = []
            if self.censura.isChecked():
                escolhidos.append("censura")
            if self.destaque.isChecked():
                escolhidos.append("destaque")
            if self.assinatura.isChecked():
                escolhidos.append("assinatura")
            return escolhidos

        def valores(self):
            return {"texto": self.texto.toPlainText().strip(), "filtros": self.filtros()}

        def mostrar_resultado(self, texto):
            self.resultado.setPlainText(texto)
```

Arquivo: `cliente/janela.py`

```
    """Janela principal do cliente grafico (PySide6).

    A janela nao conhece o dominio: todas as operacoes passam por ApiForum, o
    proxy que fala com os endpoints do servidor. Cada resposta traz o rastro
    dos padroes acionados, exibido no painel "Padroes em acao" a direita.
    """

    from PySide6.QtCore import Qt
    from PySide6.QtWidgets import (
        QComboBox,
        QHBoxLayout,
        QInputDialog,
        QLabel,
        QLineEdit,
        QListWidget,
        QListWidgetItem,
        QMainWindow,
        QMessageBox,
        QPlainTextEdit,
        QPushButton,
        QSplitter,
        QTreeWidget,
        QTreeWidgetItem,
        QVBoxLayout,
        QWidget,
    )

    from .api import ErroDaApi
    from .dialogos import (
        DialogoAnexo,
        DialogoFormatar,
        DialogoNovoTopico,
        DialogoResposta,
        DialogoUsuario,
    )

    ORDENACOES = [
        ("Mais recentes", "recentes"),
        ("Mais respondidos", "respondidos"),
        ("Mais votados", "votados"),
    ]

    ACOES_DE_ESTADO = [("Abrir", "abrir"), ("Fechar", "fechar"), ("Arquivar", "arquivar")]


    class JanelaPrincipal(QMainWindow):
        def __init__(self, api):
            super().__init__()
            self.api = api
            self._topico_atual = None
            self.setWindowTitle("Forum de Discussao - cliente Qt")
            self.resize(1180, 720)
            self._montar_interface()
            self._montar_menu()
            self.recarregar_tudo()

        # ------------------------------------------------------------------ UI --
        def _montar_interface(self):
            central = QWidget()
            principal = QVBoxLayout(central)

            # barra superior: usuario atual, ordenacao e busca
            barra = QHBoxLayout()
```

```
        barra.addWidget(QLabel("Usuario atual:"))
        self.combo_usuario = QComboBox()
        self.combo_usuario.setMinimumWidth(170)
        barra.addWidget(self.combo_usuario)

        self.botao_novo_usuario = QPushButton("Cadastrar usuario")
        self.botao_novo_usuario.clicked.connect(self.cadastrar_usuario)
        barra.addWidget(self.botao_novo_usuario)

        barra.addSpacing(18)
        barra.addWidget(QLabel("Ordenar por:"))
        self.combo_ordenacao = QComboBox()
        for rotulo, chave in ORDENACOES:
            self.combo_ordenacao.addItem(rotulo, chave)
        self.combo_ordenacao.currentIndexChanged.connect(self.carregar_topicos)
        barra.addWidget(self.combo_ordenacao)

        barra.addStretch()
        self.campo_busca = QLineEdit()
        self.campo_busca.setPlaceholderText("buscar por texto...")
        self.campo_busca.setMinimumWidth(200)
        self.campo_busca.returnPressed.connect(self.buscar)
        barra.addWidget(self.campo_busca)
        self.botao_busca = QPushButton("Buscar")
        self.botao_busca.clicked.connect(self.buscar)
        barra.addWidget(self.botao_busca)
        principal.addLayout(barra)

        divisor = QSplitter(Qt.Orientation.Horizontal)

        # coluna 1: topicos
        coluna_topicos = QWidget()
        layout_topicos = QVBoxLayout(coluna_topicos)
        layout_topicos.addWidget(QLabel("Topicos"))
        self.lista_topicos = QListWidget()
        self.lista_topicos.currentItemChanged.connect(self._mudou_topico)
        layout_topicos.addWidget(self.lista_topicos)

        self.botao_novo_topico = QPushButton("Abrir novo topico")
        self.botao_novo_topico.clicked.connect(self.abrir_topico)
        layout_topicos.addWidget(self.botao_novo_topico)

        self.botao_inscrever = QPushButton("Inscrever usuario atual")
        self.botao_inscrever.clicked.connect(self.inscrever)
        layout_topicos.addWidget(self.botao_inscrever)

        linha_estado = QHBoxLayout()
        self.combo_estado = QComboBox()
        for rotulo, chave in ACOES_DE_ESTADO:
            self.combo_estado.addItem(rotulo, chave)
        linha_estado.addWidget(self.combo_estado)
        self.botao_estado = QPushButton("Aplicar")
        self.botao_estado.clicked.connect(self.alterar_estado)
        linha_estado.addWidget(self.botao_estado)
        layout_topicos.addLayout(linha_estado)
        divisor.addWidget(coluna_topicos)

        # coluna 2: arvore da discussao
        coluna_arvore = QWidget()
        layout_arvore = QVBoxLayout(coluna_arvore)
        self.rotulo_discussao = QLabel("Discussao")
        layout_arvore.addWidget(self.rotulo_discussao)
        self.arvore = QTreeWidget()
        self.arvore.setHeaderLabels(["Item", "Autor", "Votos"])
        self.arvore.setColumnWidth(0, 330)
        self.arvore.setColumnWidth(1, 90)
        self.arvore.header().setStretchLastSection(True)
        layout_arvore.addWidget(self.arvore)

        linha_acoes = QHBoxLayout()
        self.botao_responder = QPushButton("Responder")
        self.botao_responder.clicked.connect(self.responder)
        self.botao_anexar = QPushButton("Anexar arquivo")
        self.botao_anexar.clicked.connect(self.anexar)
        self.botao_votar = QPushButton("Votar")
        self.botao_votar.clicked.connect(self.votar)
        for botao in (self.botao_responder, self.botao_anexar, self.botao_votar):
            linha_acoes.addWidget(botao)
        layout_arvore.addLayout(linha_acoes)
        divisor.addWidget(coluna_arvore)

        # coluna 3: rastro dos padroes
        coluna_rastro = QWidget()
        layout_rastro = QVBoxLayout(coluna_rastro)
        layout_rastro.addWidget(QLabel("Padroes em acao"))
        self.painel_rastro = QPlainTextEdit()
        self.painel_rastro.setReadOnly(True)
        layout_rastro.addWidget(self.painel_rastro)
        self.botao_limpar = QPushButton("Limpar painel")
        self.botao_limpar.clicked.connect(self.painel_rastro.clear)
        layout_rastro.addWidget(self.botao_limpar)
        divisor.addWidget(coluna_rastro)

        divisor.setSizes([290, 540, 350])
        principal.addWidget(divisor)
        self.setCentralWidget(central)
        self.statusBar().showMessage("Pronto.")

    def _montar_menu(self):
        menu_forum = self.menuBar().addMenu("&Forum")
```

```
        menu_forum.addAction("Atualizar", self.recarregar_tudo)
        menu_forum.addAction("Estatisticas", self.mostrar_estatisticas)
        menu_forum.addAction("Notificacoes do usuario atual", self.mostrar_notificacoes)
        menu_forum.addSeparator()
        menu_forum.addAction("Sair", self.close)

        menu_conteudo = self.menuBar().addMenu("&Conteudo")
        menu_conteudo.addAction("Formatar texto...", self.formatar_conteudo)

        menu_ajuda = self.menuBar().addMenu("&Ajuda")
        menu_ajuda.addAction("Sobre", self.mostrar_sobre)

    # ------------------------------------------------------------- apoio --
    @property
    def usuario_atual(self):
        return self.combo_usuario.currentText() or None

    def _executar(self, descricao, chamada):
        """Chama a API, trata o erro e registra o rastro no painel."""
        try:
            resposta = chamada()
        except ErroDaApi as erro:
            self._registrar(descricao, erro.rastro, falhou=True)
            QMessageBox.warning(self, "Nao foi possivel concluir", erro.mensagem)
            self.statusBar().showMessage(f"{descricao}: {erro.mensagem}", 8000)
            return None
        self._registrar(descricao, resposta.rastro)
        self.statusBar().showMessage(f"{descricao}: concluido.", 5000)
        return resposta

    def _registrar(self, descricao, rastro, falhou=False):
        marca = "FALHOU" if falhou else "OK"
        self.painel_rastro.appendPlainText(f"--- {descricao} [{marca}] ---")
        for linha in rastro or []:
            self.painel_rastro.appendPlainText(linha)
        if not rastro:
            self.painel_rastro.appendPlainText("(nenhum padrao acionado)")
        self.painel_rastro.appendPlainText("")

    def _exigir_usuario(self):
        if not self.usuario_atual:
            QMessageBox.information(
                self, "Sem usuario", "Cadastre e selecione um usuario antes desta acao."
            )
            return None
        return self.usuario_atual

    def _exigir_topico(self):
        if self._topico_atual is None:
            QMessageBox.information(self, "Sem topico", "Selecione um topico na lista.")
            return None
        return self._topico_atual

    def _item_selecionado(self):
        item = self.arvore.currentItem()
        if item is None:
            return None
        return item.data(0, Qt.ItemDataRole.UserRole)

    # ------------------------------------------------------------ carregar --
    def recarregar_tudo(self):
        self.carregar_usuarios()
        self.carregar_topicos()

    def carregar_usuarios(self):
        resposta = self._executar("Listar usuarios", self.api.listar_usuarios)
        if resposta is None:
            return
        atual = self.usuario_atual
        self.combo_usuario.clear()
        for usuario in resposta.dados:
            self.combo_usuario.addItem(usuario["apelido"], usuario)
        if atual:
            posicao = self.combo_usuario.findText(atual)
            if posicao >= 0:
                self.combo_usuario.setCurrentIndex(posicao)

    def carregar_topicos(self):
        ordenacao = self.combo_ordenacao.currentData()
        resposta = self._executar(
            f"Listar topicos ({ordenacao})", lambda: self.api.listar_topicos(ordenacao)
        )
        if resposta is None:
            return
        selecionado = self._topico_atual
        self.lista_topicos.clear()
        for topico in resposta.dados:
            item = QListWidgetItem(topico["rotulo"])
            item.setData(Qt.ItemDataRole.UserRole, topico)
            self.lista_topicos.addItem(item)
            if topico["identificador"] == selecionado:
                self.lista_topicos.setCurrentItem(item)

    def _mudou_topico(self, atual, _anterior=None):
        if atual is None:
            self._topico_atual = None
            self.arvore.clear()
            return
        self._topico_atual = atual.data(Qt.ItemDataRole.UserRole)["identificador"]
        self.carregar_discussao()
```

```
    def carregar_discussao(self):
        if self._topico_atual is None:
            return
        resposta = self._executar(
            f"Abrir discussao #{self._topico_atual}",
            lambda: self.api.obter_topico(self._topico_atual),
        )
        if resposta is None:
            return
        topico = resposta.dados
        self.rotulo_discussao.setText(
            f"{topico['titulo']} - {topico['estado']} - "
            f"{topico['total_respostas']} resposta(s), {topico['total_votos']} voto(s)"
        )
        self.arvore.clear()
        raiz = self._montar_no(topico)
        self.arvore.addTopLevelItem(raiz)
        self.arvore.expandAll()
        self.arvore.setCurrentItem(raiz)

    def _montar_no(self, dados):
        item = QTreeWidgetItem(
            [dados["rotulo"], dados["autor"], str(dados.get("votos", ""))]
        )
        item.setData(0, Qt.ItemDataRole.UserRole, dados)
        for filho in dados.get("filhos", []):
            item.addChild(self._montar_no(filho))
        return item

    # ------------------------------------------------------------------ acoes --
    def cadastrar_usuario(self):
        dialogo = DialogoUsuario(self)
        if dialogo.exec() != DialogoUsuario.DialogCode.Accepted:
            return
        valores = dialogo.valores()
        resposta = self._executar(
            "Cadastrar usuario",
            lambda: self.api.cadastrar_usuario(
                valores["apelido"], valores["email"], valores["papel"]
            ),
        )
        if resposta is None:
            return
        self.carregar_usuarios()
        posicao = self.combo_usuario.findText(resposta.dados["apelido"])
        if posicao >= 0:
            self.combo_usuario.setCurrentIndex(posicao)

    def abrir_topico(self):
        autor = self._exigir_usuario()
        if autor is None:
            return
        dados_usuario = self.combo_usuario.currentData() or {}
        dialogo = DialogoNovoTopico(self, pode_fixar=dados_usuario.get("papel") == "moderador")
        if dialogo.exec() != DialogoNovoTopico.DialogCode.Accepted:
            return
        valores = dialogo.valores()
        resposta = self._executar(
            "Abrir topico",
            lambda: self.api.abrir_topico(
                autor, valores["titulo"], valores["conteudo"], valores["tags"], valores["fixado"]
            ),
        )
        if resposta is None:
            return
        self._topico_atual = resposta.dados["identificador"]
        self.carregar_topicos()

    def responder(self):
        autor = self._exigir_usuario()
        if autor is None or self._exigir_topico() is None:
            return
        alvo = self._item_selecionado()
        if alvo is None:
            QMessageBox.information(
                self, "Sem destino", "Selecione na arvore o topico ou a mensagem a responder."
            )
            return
        dialogo = DialogoResposta(alvo["rotulo"], self)
        if dialogo.exec() != DialogoResposta.DialogCode.Accepted:
            return
        conteudo = dialogo.valores()["conteudo"]
        resposta = self._executar(
            f"Responder em {alvo['caminho']}",
            lambda: self.api.responder(alvo["identificador"], autor, conteudo),
        )
        if resposta is not None:
            self.carregar_discussao()
            self.carregar_topicos()

    def anexar(self):
        autor = self._exigir_usuario()
        if autor is None:
            return
        alvo = self._item_selecionado()
        if alvo is None or alvo["tipo"] != "mensagem":
            QMessageBox.information(
                self, "Destino invalido", "Selecione uma mensagem para receber o anexo."
            )
```

```
                return
            dialogo = DialogoAnexo(alvo["rotulo"], self)
            if dialogo.exec() != DialogoAnexo.DialogCode.Accepted:
                return
            valores = dialogo.valores()
            resposta = self._executar(
                f"Anexar em {alvo['caminho']}",
                lambda: self.api.anexar(
                    alvo["identificador"], autor, valores["nome_arquivo"], valores["tamanho_kb"]
                ),
            )
            if resposta is not None:
                self.carregar_discussao()

        def votar(self):
            alvo = self._item_selecionado()
            if alvo is None or alvo["tipo"] != "mensagem":
                QMessageBox.information(
                    self, "Destino invalido", "Somente mensagens recebem votos."
                )
                return
            resposta = self._executar(
                f"Votar em {alvo['caminho']}", lambda: self.api.votar(alvo["identificador"])
            )
            if resposta is not None:
                self.carregar_discussao()
                self.carregar_topicos()

        def inscrever(self):
            apelido = self._exigir_usuario()
            identificador = self._exigir_topico()
            if apelido is None or identificador is None:
                return
            resposta = self._executar(
                f"Inscrever {apelido} no topico #{identificador}",
                lambda: self.api.inscrever(identificador, apelido),
            )
            if resposta is not None:
                self.carregar_discussao()

        def alterar_estado(self):
            identificador = self._exigir_topico()
            if identificador is None:
                return
            acao = self.combo_estado.currentData()
            resposta = self._executar(
                f'Estado do topico #{identificador}: "{acao}"',
                lambda: self.api.alterar_estado(identificador, acao),
            )
            if resposta is not None:
                self.carregar_topicos()
                self.carregar_discussao()

        def buscar(self):
            termo = self.campo_busca.text().strip()
            if not termo:
                return
            resposta = self._executar(f'Buscar "{termo}"', lambda: self.api.buscar(termo))
            if resposta is None:
                return
            if not resposta.dados:
                QMessageBox.information(self, "Busca", f'Nenhum item contem "{termo}".')
                return
            linhas = [f"{item['caminho']} -> {item['rotulo']}" for item in resposta.dados]
            QMessageBox.information(
                self, "Busca", f"{len(linhas)} item(ns) encontrado(s):\n\n" + "\n".join(linhas)
            )

        def formatar_conteudo(self):
            dialogo = DialogoFormatar(self)

            def aplicar():
                valores = dialogo.valores()
                resposta = self._executar(
                    "Formatar conteudo",
                    lambda: self.api.formatar(
                        valores["texto"], valores["filtros"], self.usuario_atual
                    ),
                )
                if resposta is not None:
                    dialogo.mostrar_resultado(resposta.dados["texto"])

            dialogo.botao_aplicar.clicked.connect(aplicar)
            dialogo.exec()

        def mostrar_estatisticas(self):
            resposta = self._executar("Estatisticas", self.api.estatisticas)
            if resposta is None:
                return
            dados = resposta.dados
            texto = "\n".join(
                [
                    f"Usuarios cadastrados ..: {dados['usuarios']}",
                    f"Topicos abertos .......: {dados['topicos']}",
                    f"Mensagens publicadas ..: {dados['mensagens']}",
                    f"Itens na arvore .......: {dados['itens_na_arvore']}",
                    f"Votos registrados .....: {dados['votos']}",
                    f"Eventos registrados ...: {dados['eventos_registrados']}",
                    f"Proximo identificador .: {dados['proximo_identificador']}",
                    f"Instancia unica .......: {'sim' if dados['instancia_unica'] else 'nao'}",
```

```
                    ]
                )
                QMessageBox.information(self, "Estatisticas do forum", texto)

            def mostrar_notificacoes(self):
                apelido = self._exigir_usuario()
                if apelido is None:
                    return
                resposta = self._executar(
                    f"Notificacoes de {apelido}", lambda: self.api.notificacoes(apelido)
                )
                if resposta is None:
                    return
                if not resposta.dados:
                    QMessageBox.information(self, "Notificacoes", "A caixa de entrada esta vazia.")
                    return
                QMessageBox.information(self, "Notificacoes", "\n".join(resposta.dados))

            def mostrar_sobre(self):
                QMessageBox.about(
                    self,
                    "Sobre",
                    "Forum de Discussao\n\n"
                    "Projeto de Design Patterns: cliente Qt (PySide6) sobre um "
                    "servidor Flask.\nO painel 'Padroes em acao' mostra, a cada "
                    "requisicao, quais padroes o servidor acionou.",
                )


        def perguntar_endereco(pai=None, padrao="http://127.0.0.1:5000"):
            """Dialogo simples para apontar o cliente a outro endereco do servidor."""
            endereco, confirmou = QInputDialog.getText(
                pai, "Servidor", "Endereco do servidor:", text=padrao
            )
            return endereco.strip() if confirmou and endereco.strip() else None
```

Arquivo: `cliente/__main__.py`

```
        """Ponto de entrada do cliente grafico: python -m cliente [endereco]"""

        import sys

        from PySide6.QtWidgets import QApplication

        from .api import ENDERECO_PADRAO, ApiForum
        from .janela import JanelaPrincipal


        def main(argumentos=None):
            argumentos = list(sys.argv[1:] if argumentos is None else argumentos)
            endereco = argumentos[0] if argumentos else ENDERECO_PADRAO

            aplicacao = QApplication(sys.argv[:1])
            janela = JanelaPrincipal(ApiForum(endereco))
            janela.show()
            return aplicacao.exec()


        if __name__ == "__main__":
            sys.exit(main())
```

---

8. Testes automatizados

A suíte cobre as três camadas exigidas: o código do servidor (domínio e rotas), o código do cliente (acesso à API e interface) e o comportamento de ambos diante de entradas inválidas e de regras de negócio violadas.

```
python -m unittest discover -s testes -v
```

Arquivo	Grupo	Testes	O que cobre
`test_padroes.py`	TesteSingleton	3	Instância única, estado compartilhado e contador contínuo
	TesteFactoryMethod	3	Produto de cada criador e tipo desconhecido
	TesteBuilder	3	Construção encadeada, campos obrigatórios e reinício
	TesteComposite	5	Contagem recursiva, caminho, serialização, indentação e folha sem filhos
	TesteDecorator	3	Censura, pilha de filtros e efeito da ordem
	TesteObserver	3	Notificação, inscrição duplicada e cancelamento
	TesteStrategy	3	Cada ordenação e troca em execução
	TesteState	3	Estado inicial, bloqueio por fechamento e estado terminal
	TesteFacade	7	Fluxo completo, erros com código HTTP e rastro dos padrões

`test_servidor.py`	TesteIndiceESaude	4	Índice, saúde, rota inexistente e método não permitido
	TesteEnvelope	2	Formato da resposta e isolamento do rastro entre requisições
	TesteUsuarios	5	Cadastro, listagem, duplicidade, campos inválidos e usuário inexistente
	TesteTopicos	8	Abertura, permissão de fixar, ordenações e tópico inexistente
	TesteRespostasEAnexos	6	Aninhamento, anexo em folha, resposta vazia e voto em tópico
	TesteEstadoEObserver	5	Inscrição, notificação, fechamento e estado terminal
	TesteConteudoEConsultas	5	Pilha de filtros, filtro inválido, busca e estatísticas
`test_cliente.py`	TesteTransporte	6	Montagem da rota, envelope, erro HTTP, servidor fora do ar e resposta não-JSON
	TesteOperacoes	6	Fluxo de discussão, ordenação, estado, formatação, notificações e busca
`test_cliente_gui.py`	TesteJanelaPrincipal	10	Carga inicial, árvore, painel de padrões, ações e falha de comunicação

Os testes do cliente conversam com o servidor real por meio de uma sessão de teste que encaminha cada chamada ao `test_client` do Flask: o caminho completo (cliente → HTTP → rota → fachada → domínio) é exercitado sem abrir porta de rede. Os testes de interface usam a plataforma `offscreen` do Qt e são ignorados automaticamente se o PySide6 não estiver instalado.

Resultado da execução:

```
test_busca (test_cliente.TesteOperacoes.test_busca) ... ok
test_estado_e_recusa_de_resposta (test_cliente.TesteOperacoes.test_estado_e_recusa_de_resposta) ... ok
test_fluxo_de_discussao (test_cliente.TesteOperacoes.test_fluxo_de_discussao) ... ok
test_formatacao (test_cliente.TesteOperacoes.test_formatacao) ... ok
test_listagem_com_ordenacao (test_cliente.TesteOperacoes.test_listagem_com_ordenacao) ... ok
test_notificacoes_e_estatisticas (test_cliente.TesteOperacoes.test_notificacoes_e_estatisticas) ... ok
test_endereco_sem_barra_final (test_cliente.TesteTransporte.test_endereco_sem_barra_final) ... ok
test_erro_http_vira_excecao (test_cliente.TesteTransporte.test_erro_http_vira_excecao) ... ok
test_monta_a_rota_correta (test_cliente.TesteTransporte.test_monta_a_rota_correta) ... ok
test_resposta_que_nao_e_json (test_cliente.TesteTransporte.test_resposta_que_nao_e_json) ... ok
test_resposta_traz_dados_e_rastro (test_cliente.TesteTransporte.test_resposta_traz_dados_e_rastro) ... ok
test_servidor_fora_do_ar (test_cliente.TesteTransporte.test_servidor_fora_do_ar) ... ok
test_alterar_estado_pela_janela (test_cliente_gui.TesteJanelaPrincipal.test_alterar_estado_pela_janela) ... ok
test_arvore_reflete_a_estrutura_recebida (test_cliente_gui.TesteJanelaPrincipal.test_arvore_reflete_a_estrutura_recebida) ... ok
test_carrega_usuarios_no_seletor (test_cliente_gui.TesteJanelaPrincipal.test_carrega_usuarios_no_seletor) ... ok
test_falha_de_servidor_mostra_aviso (test_cliente_gui.TesteJanelaPrincipal.test_falha_de_servidor_mostra_aviso) ... ok
test_inscrever_usuario_atual (test_cliente_gui.TesteJanelaPrincipal.test_inscrever_usuario_atual) ... ok
test_lista_topicos_apos_recarregar (test_cliente_gui.TesteJanelaPrincipal.test_lista_topicos_apos_recarregar) ... ok
test_lista_vazia_quando_nao_ha_topicos (test_cliente_gui.TesteJanelaPrincipal.test_lista_vazia_quando_nao_ha_topicos) ... ok
test_painel_registra_os_padroes (test_cliente_gui.TesteJanelaPrincipal.test_painel_registra_os_padroes) ... ok
test_votar_em_topico_e_recusado_pelo_servidor (test_cliente_gui.TesteJanelaPrincipal.test_votar_em_topico_e_recusado_pelo_servidor) ... ok
test_votar_sem_selecao_avisa (test_cliente_gui.TesteJanelaPrincipal.test_votar_sem_selecao_avisa) ... ok
test_builder_incompleto (test_padroes.TesteBuilder.test_builder_incompleto) ... ok
test_construcao_encadeada (test_padroes.TesteBuilder.test_construcao_encadeada) ... ok
test_reinicia_apos_construir (test_padroes.TesteBuilder.test_reinicia_apos_construir) ... ok
test_caminho_ate_a_raiz (test_padroes.TesteComposite.test_caminho_ate_a_raiz) ... ok
test_contagem_recursiva (test_padroes.TesteComposite.test_contagem_recursiva) ... ok
test_folha_nao_aceita_filhos (test_padroes.TesteComposite.test_folha_nao_aceita_filhos) ... ok
test_linhas_indentadas (test_padroes.TesteComposite.test_linhas_indentadas) ... ok
test_serializacao_recursiva (test_padroes.TesteComposite.test_serializacao_recursiva) ... ok
test_censura (test_padroes.TesteDecorator.test_censura) ... ok
test_ordem_importa (test_padroes.TesteDecorator.test_ordem_importa) ... ok
test_pilha_de_decoradores (test_padroes.TesteDecorator.test_pilha_de_decoradores) ... ok
test_busca_recursiva (test_padroes.TesteFacade.test_busca_recursiva) ... ok
test_fluxo_completo (test_padroes.TesteFacade.test_fluxo_completo) ... ok
test_item_inexistente (test_padroes.TesteFacade.test_item_inexistente) ... ok
test_rastro_registra_os_padroes (test_padroes.TesteFacade.test_rastro_registra_os_padroes) ... ok
test_somente_moderador_fixa (test_padroes.TesteFacade.test_somente_moderador_fixa) ... ok
test_topico_fechado_recusa_resposta (test_padroes.TesteFacade.test_topico_fechado_recusa_resposta) ... ok
test_usuario_duplicado (test_padroes.TesteFacade.test_usuario_duplicado) ... ok
test_cada_criador_devolve_seu_produto (test_padroes.TesteFactoryMethod.test_cada_criador_devolve_seu_produto) ... ok
test_criador_por_nome (test_padroes.TesteFactoryMethod.test_criador_por_nome) ... ok
test_tipo_desconhecido (test_padroes.TesteFactoryMethod.test_tipo_desconhecido) ... ok
test_desinscrever (test_padroes.TesteObserver.test_desinscrever) ... ok
test_nao_duplica_inscricao (test_padroes.TesteObserver.test_nao_duplica_inscricao) ... ok
test_notifica_inscritos (test_padroes.TesteObserver.test_notifica_inscritos) ... ok
test_contador_nao_reinicia (test_padroes.TesteSingleton.test_contador_nao_reinicia) ... ok
test_estado_compartilhado (test_padroes.TesteSingleton.test_estado_compartilhado) ... ok
test_instancia_unica (test_padroes.TesteSingleton.test_instancia_unica) ... ok
test_arquivado_e_terminal (test_padroes.TesteState.test_arquivado_e_terminal) ... ok
test_estado_inicial (test_padroes.TesteState.test_estado_inicial) ... ok
test_fechar_bloqueia_respostas (test_padroes.TesteState.test_fechar_bloqueia_respostas) ... ok
test_mais_recentes (test_padroes.TesteStrategy.test_mais_recentes) ... ok
test_mais_respondidos (test_padroes.TesteStrategy.test_mais_respondidos) ... ok
test_troca_em_execucao (test_padroes.TesteStrategy.test_troca_em_execucao) ... ok
test_busca_encontra_na_arvore (test_servidor.TesteConteudoEConsultas.test_busca_encontra_na_arvore) ... ok
test_busca_sem_termo (test_servidor.TesteConteudoEConsultas.test_busca_sem_termo) ... ok
test_estatisticas (test_servidor.TesteConteudoEConsultas.test_estatisticas) ... ok
test_filtro_desconhecido (test_servidor.TesteConteudoEConsultas.test_filtro_desconhecido) ... ok
test_formatacao_com_pilha_de_filtros (test_servidor.TesteConteudoEConsultas.test_formatacao_com_pilha_de_filtros) ... ok
```

```
test_rastro_nao_vaza_entre_requisicoes (test_servidor.TesteEnvelope.test_rastro_nao_vaza_entre_requisicoes) ... ok
test_resposta_traz_dados_e_rastro (test_servidor.TesteEnvelope.test_resposta_traz_dados_e_rastro) ... ok
test_acao_de_estado_invalida (test_servidor.TesteEstadoEObserver.test_acao_de_estado_invalida) ... ok
test_arquivado_nao_reabre (test_servidor.TesteEstadoEObserver.test_arquivado_nao_reabre) ... ok
test_fechar_bloqueia_resposta (test_servidor.TesteEstadoEObserver.test_fechar_bloqueia_resposta) ... ok
test_inscricao_duplicada (test_servidor.TesteEstadoEObserver.test_inscricao_duplicada) ... ok
test_inscricao_e_notificacao (test_servidor.TesteEstadoEObserver.test_inscricao_e_notificacao) ... ok
test_indice_lista_endpoints (test_servidor.TesteIndiceESaude.test_indice_lista_endpoints) ... ok
test_metodo_nao_permitido (test_servidor.TesteIndiceESaude.test_metodo_nao_permitido) ... ok
test_rota_inexistente (test_servidor.TesteIndiceESaude.test_rota_inexistente) ... ok
test_saude (test_servidor.TesteIndiceESaude.test_saude) ... ok
test_anexo_em_anexo_e_recusado (test_servidor.TesteRespostasEAnexos.test_anexo_em_anexo_e_recusado) ... ok
test_anexo_em_mensagem (test_servidor.TesteRespostasEAnexos.test_anexo_em_mensagem) ... ok
test_resposta_a_item_inexistente (test_servidor.TesteRespostasEAnexos.test_resposta_a_item_inexistente) ... ok
test_resposta_aninhada (test_servidor.TesteRespostasEAnexos.test_resposta_aninhada) ... ok
test_resposta_vazia (test_servidor.TesteRespostasEAnexos.test_resposta_vazia) ... ok
test_voto_em_topico_e_recusado (test_servidor.TesteRespostasEAnexos.test_voto_em_topico_e_recusado) ... ok
test_abertura_devolve_arvore (test_servidor.TesteTopicos.test_abertura_devolve_arvore) ... ok
test_autor_inexistente (test_servidor.TesteTopicos.test_autor_inexistente) ... ok
test_membro_nao_fixa (test_servidor.TesteTopicos.test_membro_nao_fixa) ... ok
test_moderador_fixa (test_servidor.TesteTopicos.test_moderador_fixa) ... ok
test_ordenacao_invalida (test_servidor.TesteTopicos.test_ordenacao_invalida) ... ok
test_ordenacao_por_votos (test_servidor.TesteTopicos.test_ordenacao_por_votos) ... ok
test_titulo_vazio (test_servidor.TesteTopicos.test_titulo_vazio) ... ok
test_topico_inexistente (test_servidor.TesteTopicos.test_topico_inexistente) ... ok
test_apelido_duplicado (test_servidor.TesteUsuarios.test_apelido_duplicado) ... ok
test_apelido_vazio (test_servidor.TesteUsuarios.test_apelido_vazio) ... ok
test_cadastro_e_listagem (test_servidor.TesteUsuarios.test_cadastro_e_listagem) ... ok
test_notificacoes_de_usuario_inexistente (test_servidor.TesteUsuarios.test_notificacoes_de_usuario_inexistente) ... ok
test_papel_invalido (test_servidor.TesteUsuarios.test_papel_invalido) ... ok

----------------------------------------------------------------------
Ran 90 tests in 0.726s

OK
```

---

9. Demonstração da aplicação em uso

O roteiro abaixo é produzido por `testes/demonstracao.py`, que sobe o servidor em uma porta própria, executa o cenário completo pelos endpoints — exatamente como o cliente faz — e imprime, a cada passo, a chamada e o rastro de padrões devolvido pelo servidor. Ao final, abre a janela do cliente ligada a esse servidor e salva as capturas de tela usadas na seção 6.

```
python testes/demonstracao.py
```

Saída da execução:

```
Servidor de demonstracao em http://127.0.0.1:5055

======================================================================
PASSO 1 - Cadastro de usuarios
======================================================================

> POST /api/usuarios (ana, membro)
  rastro | [Facade] Usuario "ana" gravado no repositorio unico

> POST /api/usuarios (bruno, moderador)
  rastro | [Facade] Usuario "bruno" gravado no repositorio unico

> POST /api/usuarios (apelido repetido)
  resposta: HTTP 409 - Ja existe um usuario com o apelido "ana".

======================================================================
PASSO 2 - Abertura de topicos (Builder + Factory Method)
======================================================================

> POST /api/topicos (ana)
  rastro | [Facade] Coordenando Builder + Factory Method + Singleton + Observer
  rastro | [Builder] Montando o topico passo a passo:
  rastro |   - titulo ...: Como funciona a recursao em Python?
  rastro |   - autor ....: ana
  rastro |   - conteudo .: Nao entendi o caso base.
  rastro |   - tags .....: python, recursao
  rastro |   - fixado ...: nao
  rastro | [Factory Method] CriadorDeTopico vai fabricar um(a) Topico
  rastro | [Factory Method] Produto criado: [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
  rastro | [Facade] Topico guardado em /T1 e observadores inscritos

> POST /api/topicos (membro tentando fixar)
  resposta: HTTP 403 - Somente moderadores podem fixar um topico.

> POST /api/topicos (moderador fixando)
  rastro | [Facade] Coordenando Builder + Factory Method + Singleton + Observer
  rastro | [Builder] Montando o topico passo a passo:
  rastro |   - titulo ...: Regras do forum
  rastro |   - autor ....: bruno
  rastro |   - conteudo .: Leia antes de postar.
  rastro |   - tags .....: avisos
  rastro |   - fixado ...: sim
  rastro | [Factory Method] CriadorDeTopico vai fabricar um(a) Topico
```

```
  rastro | [Factory Method] Produto criado: [TOPICO #2] Regras do forum <Aberto> - por bruno (tags: avisos)
  rastro | [Facade] Topico guardado em /T2 e observadores inscritos

======================================================================
PASSO 3 - Inscricao e resposta (Observer + State + Composite)
======================================================================

> POST /api/topicos/1/inscricoes (bruno)
  rastro | [Observer] "bruno" inscrito. Total de observadores: 3

> POST /api/itens/1/respostas (bruno responde ao topico)
  rastro | [Facade] Consultando o estado do topico #1
  rastro | [Factory Method] CriadorDeMensagem vai fabricar um(a) Mensagem
  rastro | [Factory Method] Produto criado: [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 0)
  rastro | [Composite] Mensagem inserida em /T1/M3
  rastro | [Observer] Notificando 3 inscrito(s) sobre o evento "nova-resposta"
  rastro |   -> caixa de entrada de ana: bruno respondeu em "Como funciona a recursao em Python?"
  rastro |   -> registro de atividade: nova-resposta gravado (total: 1)
  rastro |   -> caixa de entrada de bruno: bruno respondeu em "Como funciona a recursao em Python?"

> POST /api/itens/3/respostas (ana responde a mensagem #3)
  rastro | [Facade] Consultando o estado do topico #1
  rastro | [Factory Method] CriadorDeMensagem vai fabricar um(a) Mensagem
  rastro | [Factory Method] Produto criado: [MSG #4] ana: Obrigada! Funcionou aqui. (votos: 0)
  rastro | [Composite] Mensagem inserida em /T1/M3/M4
  rastro | [Observer] Notificando 3 inscrito(s) sobre o evento "nova-resposta"
  rastro |   -> caixa de entrada de ana: ana respondeu em "Como funciona a recursao em Python?"
  rastro |   -> registro de atividade: nova-resposta gravado (total: 2)
  rastro |   -> caixa de entrada de bruno: ana respondeu em "Como funciona a recursao em Python?"

> POST /api/mensagens/4/anexos (folha da arvore)
  rastro | [Factory Method] CriadorDeAnexo vai fabricar um(a) Anexo
  rastro | [Factory Method] Produto criado: [ANEXO #5] saida.txt (2 KB)
  rastro | [Composite] Folha inserida em /T1/M3/M4/A5

======================================================================
PASSO 4 - Arvore da discussao (Composite serializado)
======================================================================

> GET /api/topicos/1
  rastro | [Composite] Percorrendo a arvore a partir de /T1

  arvore recebida pelo cliente:
  [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
     [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 0)
        [MSG #4] ana: Obrigada! Funcionou aqui. (votos: 0)
           [ANEXO #5] saida.txt (2 KB)

======================================================================
PASSO 5 - Ordenacao da listagem (Strategy)
======================================================================

> POST /api/mensagens/3/votos
  rastro | Voto registrado em /T1/M3 (total: 1)

> POST /api/mensagens/3/votos (segundo voto)
  rastro | Voto registrado em /T1/M3 (total: 2)

> GET /api/topicos?ordenacao=recentes
  rastro | [Strategy] Estrategia mantida: mais recentes
  rastro | [Strategy] Ordenando os topicos por: mais recentes
  1. [TOPICO #2] [FIXADO] Regras do forum <Aberto> - por bruno (tags: avisos) (respostas: 0, votos: 0)
  2. [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao) (respostas: 2, votos: 2)

> GET /api/topicos?ordenacao=votados
  rastro | [Strategy] Estrategia trocada: mais recentes -> mais votados
  rastro | [Strategy] Ordenando os topicos por: mais votados
  1. [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao) (respostas: 2, votos: 2)
  2. [TOPICO #2] [FIXADO] Regras do forum <Aberto> - por bruno (tags: avisos) (respostas: 0, votos: 0)

======================================================================
PASSO 6 - Ciclo de vida do topico (State)
======================================================================

> PUT /api/topicos/1/estado (fechar)
  rastro | [Facade] Pedindo "fechar" ao estado atual (Aberto)
  rastro | [State] Transicao: Aberto -> Fechado

> POST /api/itens/1/respostas (topico fechado)
  resposta: HTTP 409 - O topico esta Fechado e nao aceita novas respostas.
  rastro | [Facade] Consultando o estado do topico #1

> PUT /api/topicos/1/estado (arquivar)
  rastro | [Facade] Pedindo "arquivar" ao estado atual (Fechado)
  rastro | [State] Transicao: Fechado -> Arquivado

> PUT /api/topicos/1/estado (reabrir arquivado)
  resposta: HTTP 409 - Nao e possivel "abrir" um topico Arquivado.
  rastro | [Facade] Pedindo "abrir" ao estado atual (Arquivado)
  rastro | [State] Transicao recusada: nao e possivel "abrir" um topico Arquivado

======================================================================
PASSO 7 - Formatacao de conteudo (Decorator)
======================================================================

> POST /api/conteudo/formatado (censura + destaque + assinatura)
  rastro | [Decorator] Pilha aplicada (de dentro para fora): censura -> destaque -> assinatura

  texto renderizado:
  >>> Esse post e um ****, autor *****. <<<
```

```
  -- bruno (moderador)

======================================================================
PASSO 8 - Busca, notificacoes e estatisticas
======================================================================

> GET /api/busca?termo=recursao
  rastro | [Facade] Buscando "recursao" em todas as arvores de discussao
  rastro | 2 item(ns) encontrado(s)
  /T1 -> [TOPICO #1] Como funciona a recursao em Python? <Arquivado> - por ana (tags: python, recursao)
  /T1/M3 -> [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 2)

> GET /api/usuarios/ana/notificacoes
  [nova-resposta] bruno respondeu em "Como funciona a recursao em Python?"
  [nova-resposta] ana respondeu em "Como funciona a recursao em Python?"

> GET /api/estatisticas
  rastro | [Singleton] Forum() devolve o mesmo objeto? True
  eventos_registrados = 2
  instancia_unica = True
  itens_na_arvore = 5
  mensagens = 2
  proximo_identificador = 6
  topicos = 2
  usuarios = 2
  votos = 2

Demonstracao concluida.
```